

Microcontroladores

Semana 4

Semestre 2026-2
Por Kalun José Lau Gan

Preguntas previas

- En el ejercicio de generación de retardo de 40ms de la semana 3. ¿Cómo se declaran las etiquetas de los GPR?
 - En la parte superior del programa debajo de la declaración del PSECT se coloca:


```
x_var EQU 500H
y_var EQU 501H
```
- Hay una nueva versión del XC8: **v2.50 (actualizado al 02/09/2024)**
- ¿Cómo puedo saber en que banco se encuentra un registro?
 - Revisando en la hoja técnica, generalmente los registros de manipulación de E/S se encuentran en el BANK4, los registros que manipulan la configuración del módulo de oscilador en el BANK0, la memoria RAM esta mapeada a partir del BANK5 y así en sucesivo.
 - En el detalle de algún registro, se visualiza la dirección por ejemplo 0x04C7, debido a que el BSR ocupa los bits del 8 al 13, entonces el bank de dicho registro será 04.

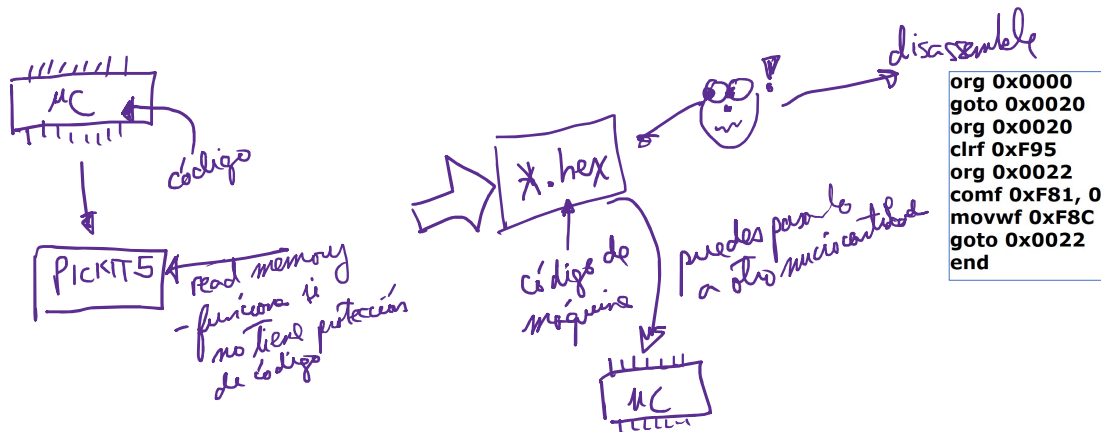
04C7H
BSR

MOVLB	k	Move literal to BSR<5:0>
-------	---	--------------------------

0x04C7	TRISB	7:0
--------	-------	-----

Preguntas previas 2024:

- Profe, quiero sacar el programa de un micro para ver que hace y luego pasarlo a otro micro pero no se entiende nada.

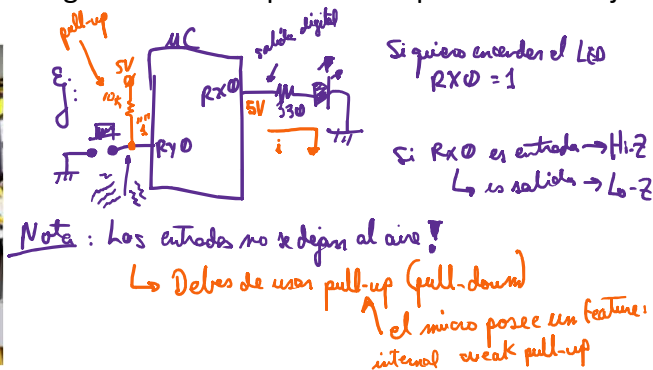


Preguntas previas 2024:

- Profe, he querido hacer los circuitos que hemos hecho en clase durante el fin de semana y no funciona, que puede ser?
 - Hay tres etapas importantes:
 - Durante el desarrollo del código en el MPLABX, si el código no compila revisar sintaxis. Recordar que si no se logra compilar, no se generan los archivos para grabar el microcontrolador.
 - Cuando se esta en el evento de programar la memoria del μC no sale el mensaje "Programming Successful", posiblemente es problema del hardware, posiblemente falla en la conexión entre la PC y el Curiosity Nano, o algún evento de corto circuito que no permite energizar y hacer la programación
 - Cuando se programó satisfactoriamente el μC y no funciona la aplicación, problemas en la implementación del hardware, revisar polaridad de los LEDs, valor de las resistencias, continuidad de los cables jumper, conexiones de alimentación, protoboard en mal estado, componentes quemados, rieles de alimentación del protoboard sin energía, etc.

Preguntas previas 2024:

- Profe, cuando pruebo los ejercicios no funciona la implementación, acerco mi mano y si funciona.
 - Tienes manos "mágicas" ... XD
 - El cuerpo humano tiene una capacitancia, esto permite que puedas almacenar carga eléctrica y que eso se ve reflejado cuando acerca la mano al circuito.
 - Revisar configuración de los puertos empleados en el ejemplo.

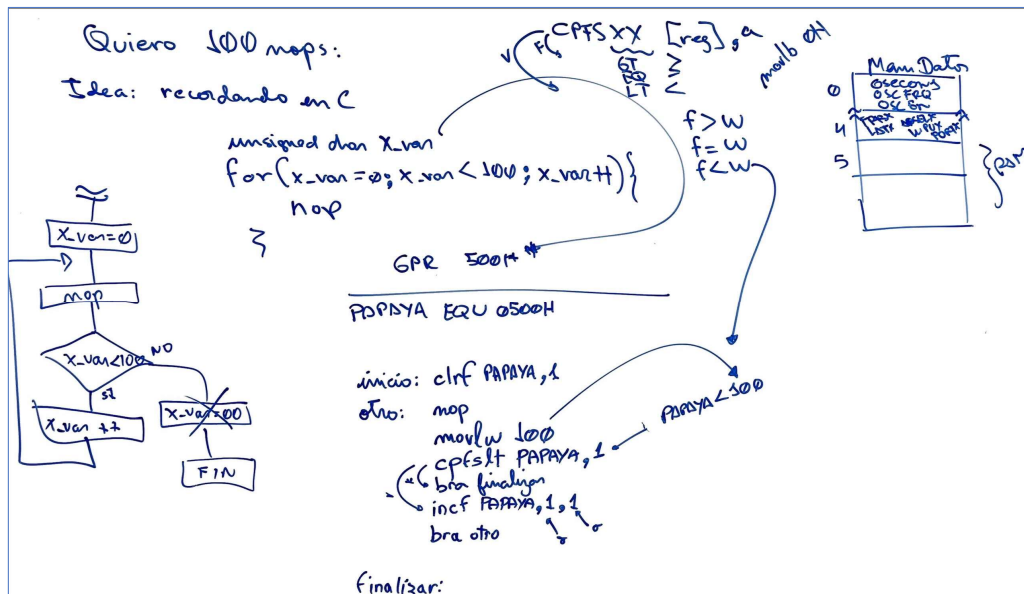


NOTA:

Recordar que los registros para manipular los E/S se encuentran en el BANK4 (revisar Cap 19 del datasheet)

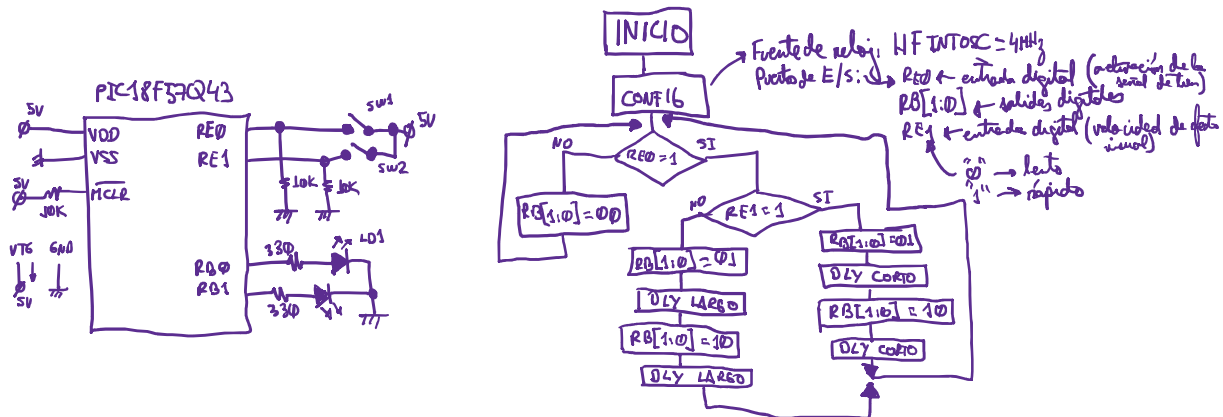
BANK4
 $WPUB.4 = 1$

Preguntas previas 2024:



Preguntas previas 2024-2

- ¿Cómo sería la estructura funcional (diagrama de flujo) de la asignación que se dejó planteado en la semana 3?



No cometer estas acciones durante las evaluaciones

```

Everyone
Diego Luis Andres Estrella R... 4:05 PM
PROCESSOR 18F57Q43
#include "cabecera.inc"

PSECT upcino, class=CODE, relleno2,
abs

upcino:
ORG 000000H
bra configur

ORG 001000H
tabla_7s DB 3FH, 0EH, 5BH, 4FH, 5BH,
6DH, 7DH, 07H, 7FH, 67H, 4DH, 5BH,
5CH, 7CH, 5BH, 75H

ORG 000100H

configur:
movlw 0H
movlw 5DH
movlw OSCCON1, 1
movlw 03H
movlw OSCFRQ, 1
movlw 40H
movlw OSCEN, 1
movlw 4H
movlw 8DH

movlw TRISB, 1
movlw ANSELB, 1
movlw 0DH
movlw ANSELB, 1
movlw WPU, 1

movlw 0FH
cif TBLPTRU, 1
movlw 1DH
movlw TBLPTRH, 1
cif TBLPTRL, 1
  
```

PROBLEMAS CON EL LAB 1 GRUPO 6

u202013651 (Villanueva Gonzales, Sebastian Alonso)
To: pcelikau (Lau Gan, Kalun Jose) Kalun José Lau Gan Wed 4/10/2024 21:17

Estimado Profesor,

Habiendo acabado el laboratorio 1, mi grupo ha notado que accidentalmente subimos el archivo al grupo 6 pero de otra sección. Como puede visualizar en esta imagen (y en la carpeta compartida) el archivo fue subido a tiempo. Por la presión del tiempo finalmente erramos en la carpeta. Espero sepa comprender nuestro error para la calificación. Los integrantes de mi grupo son Villanueva Gonzales, Sebastian Alonso - u202013651, Paredes Eche, Alessandro - u202112055, Espichan Quispe, Alci Mateo - u202111525, Hernandez De La Cruz, Diego Sebastian - u202123436 de la sección EL51 en el grupo 6.

Mis mejores deseos,

Villanueva Gonzales, Sebastian Alonso



Preguntas previas 2025-2

- ¿Cuáles son las herramientas oficiales actuales para programar un microcontrolador PIC?

- El PICKit 5
- El ICD 5
- El MPLAB Snap
- El ICE 4



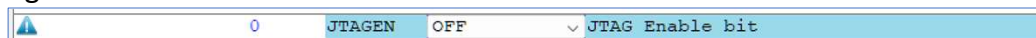
Preguntas previas 2025-2

- ¿Cuáles son las diferencias principales entre la familia Q43 y la familia Q84 de los PIC18 de Microchip?

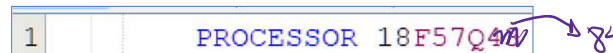
- En el 57Q84:
 - Mayor memoria RAM (12800 bytes)
 - Soporte para comunicación CAN
 - Posee un Timer universal

- He conseguido el Curiosity Nano PIC18F57Q84 y no se como se configura para los ejemplos del curso.

- En propiedades del proyecto debe de estar seleccionado como dispositivo PIC18F57Q84 y como herramienta (tool) el Curiosity Nano PIC18F57Q84.
- En los bits de configuración deshabilitar el JTAG además de los cinco bits de configuración hechos en el Q43.

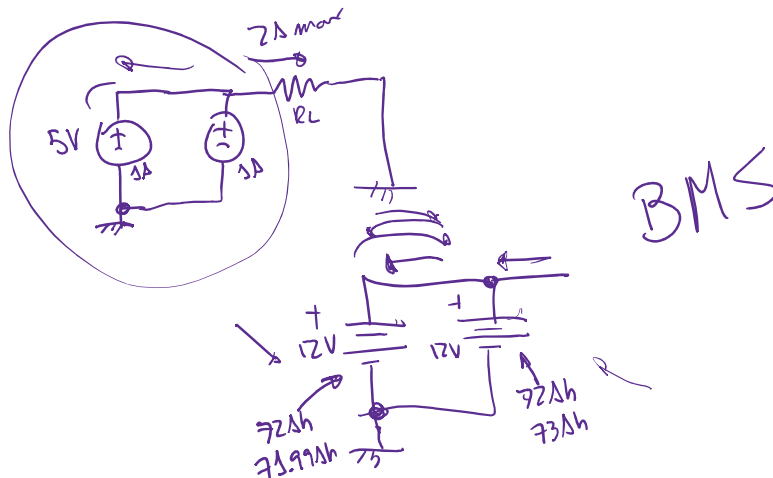


- En el código fuente colocar en la directiva PROCESSOR el 18F57Q84



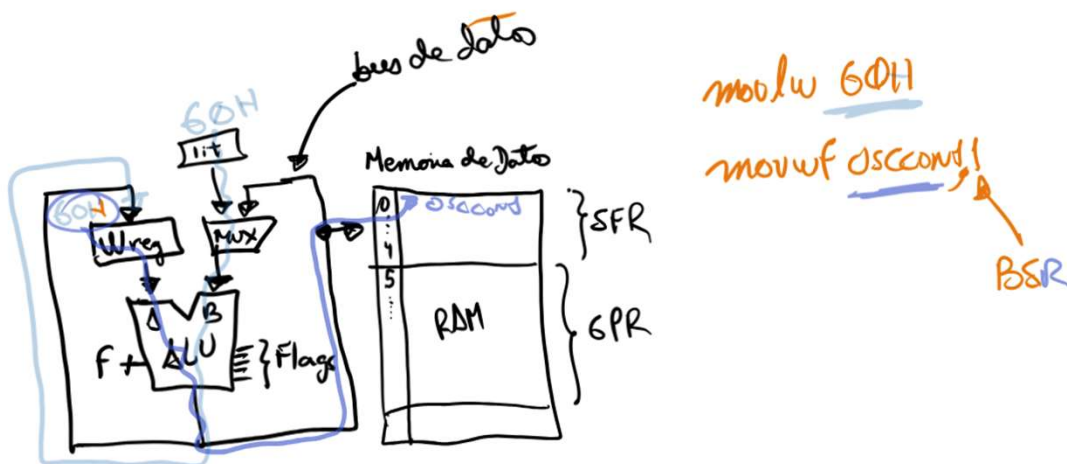
Atención de preguntas 2026-1

- ¿Puedo conectar dos fuentes o dos pilas en paralelo?
- NO!



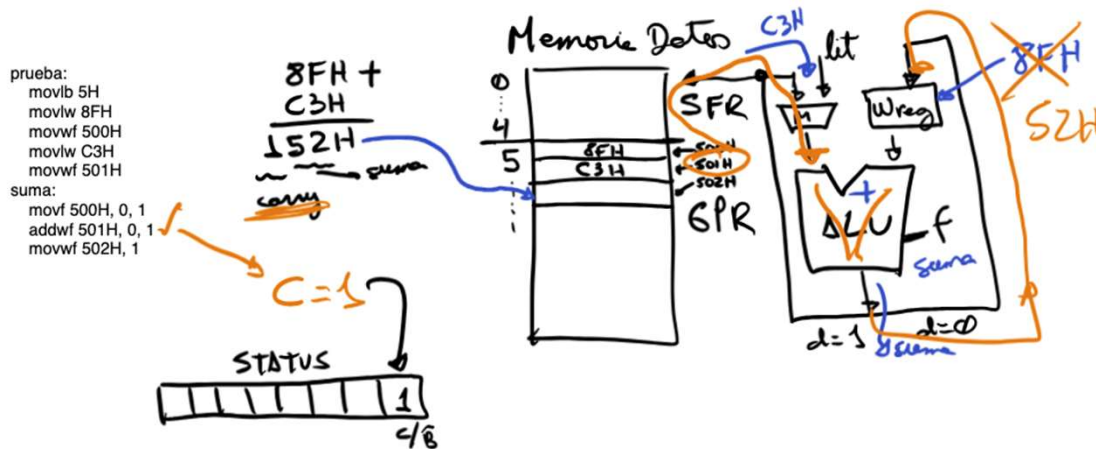
Atención de preguntas 2026-1

- Cómo funciona el registro W?



Atención de preguntas 2026-1

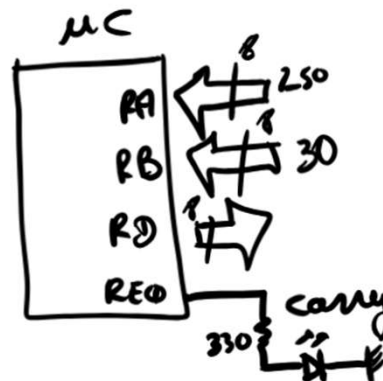
- Puede explicarnos como hace una operacion aritmetica de suma de dos digitos y su bandera de carry?



Atención de preguntas 2026-1

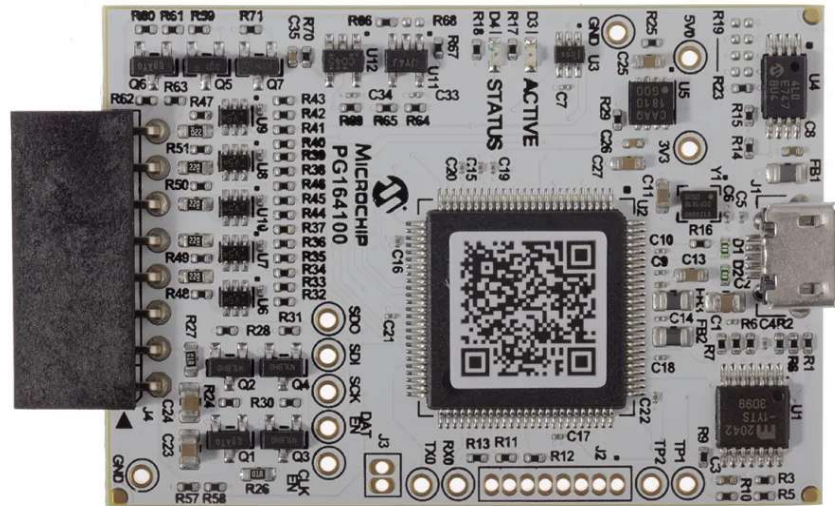
- Se puede mandar la señal de carry hacia un LED en un puerto?

inicio:
 movf PORTA, 0, 1
 addwf PORTB, 0, 1
 btfss STATUS, 0, 1
 bra no_carry
 bsf LATE, 0, 1
 bra sigue
 no_carry:
 bcf LATE, 0, 1
 sigue:
 movwf LATD, 1
 bra inicio



Detalle del MPLAB SNAP

- Pendiente de verificación si es que el pin 5V0 proviene del mismo puerto micro USB del módulo

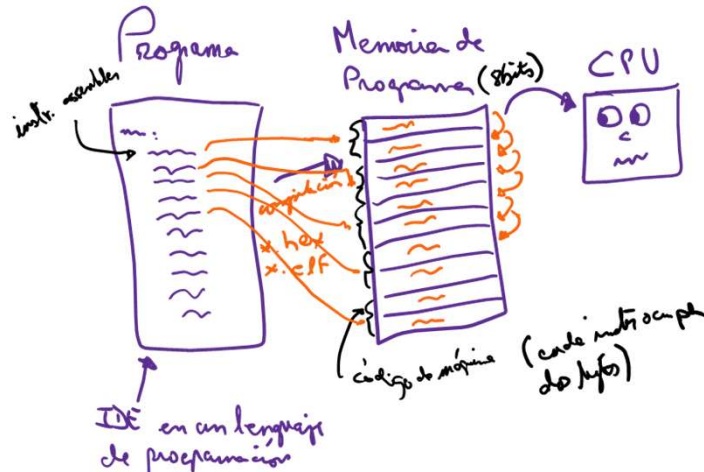


Agenda

- El contador de programa
- Los displays de siete segmentos
- Datos constantes en la memoria de programa
- El puntero de tabla (TBLPTR)
- Instrucciones de comparación numérica

Recordando:

- La ejecución de un programa en el microcontrolador es de manera **secuencial** (una instrucción a la vez) y **ordenada** (uno detrás de otro).



La ejecución es ordenada y secuencial

inicio:	;etiqueta llamada inicio
1° → bsf LATF, 6, 1	;RF6 en uno lógico
2° → nop	;retardo de 1us si Fosc=4MHz
3° → bcf LATF, 6, 1	;RF6 en cero lógico
4° → nop	;retardo de 1us si Fosc=4MHz
5° → bra inicio	;salto a etiqueta inicio

Aclaración sobre ejecución secuencial ante instrucciones de toma de decisión

```

inicio:
  nop
  nop
  btfss PORTE, 6, 1    ;pregunta si el bit 6 del PORTE es uno

```

```

  bra es_falso
  bra es_cierto

```

;dejar espacios no significa que el CPU va a capturarlos y ejecutarlos como línea, lo que va a pasar es que al momento de hacer la compilación, el compilador simplemente elimina dichos espacios

Cuando el programa anterior se pasa a la memoria de programa del microcontrolador

Memoria de programa (8 bit o 1 byte)	
Contenido	Dirección
bsf LATF, 6, 1 (4CH)	000000H
nop	000001H
	000002H
btfss LATF, 6, 1 (4CH)	000003H
	000004H
nop	000005H
	000006H
	000007H
bra inicio (000000H)	000008H
	000009H
	00000AH
	00000BH

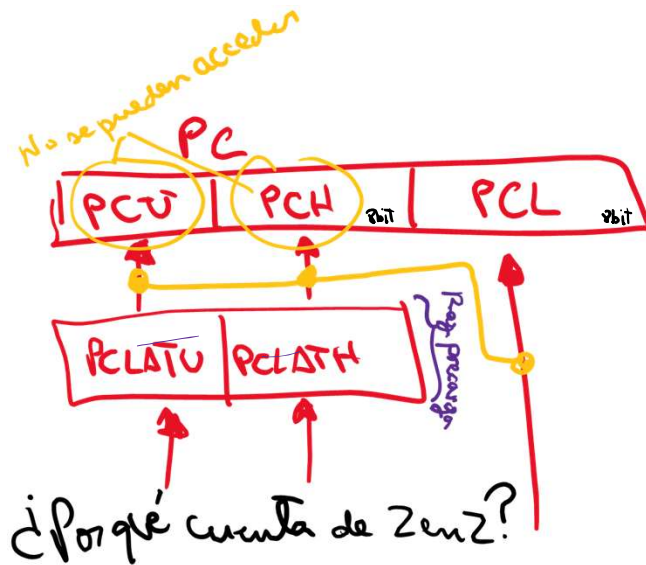
Diagrama de flujo de ejecución:

- 1° → 2°
- 2° → 3°
- 3° → 4°
- 4° → 5°
- 5° → 1° (bucle)

Etiquetas de tiempo:

- t2
- t2
- t2
- t2

El Contador de Programa (PC)

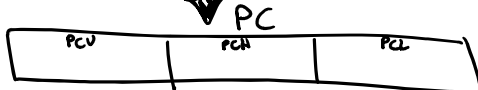


- Indispensable para la ejecución secuencial de las instrucciones.
- Su función es de alojar la dirección de la siguiente instrucción que el CPU va a ejecutar.
- Cuenta de dos en dos
- Según hoja técnica, consta de 21 bits separados en tres registros: PCU, PCH y PCL.
- PCU y PCH no son accesibles, para que se pueda escribir la dirección de 21bits es necesario seguir paso previo que es la de precargar los datos en los registros previos PCLATH y PCLATH, y solamente se transferirán hacia PCU y PCH respectivamente cuando PCL le cargues un dato.
- Al escribir en PCL se subirán PCLATH y PCLATU para así subir los 21 bits a la vez
- En el PIC18F47Q10, se tiene solamente 17 bits ya que la memoria es de 128Kbyte

Ejemplo

- Cargar dirección **00288AH** en PC:

clr f PCLATU ← Precarga
movlw 28H
movwf PCLATH ← Precarga
movlw 8AH
movwf PCL ← Se carga PCL
 PCLATH hacia el PC



NOTA: El banco activo debe de ser el 4

Address	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00	Reserved									
0x0FD7	STATUS	7:0		TO	PD	N	OV	Z	DC	C
0x0FD9	FSR2	7:0								
0x0FDB	PLUSW2	15:8								
0x0FDC	PREINC2	7:0								
0x0FDD	POSTDEC2	7:0								
0x0FDE	POSTINC2	7:0								
0x0FDF	INDR2	7:0								
0x0FE0	BSR	7:0								
0x0FE1	FSR1	15:8								
0x0FE3	PLUSW1	7:0								
0x0FE4	PREINC1	7:0								
0x0FE5	POSTDEC1	7:0								
0x0FE6	POSTINC1	7:0								
0x0FE7	INDR1	7:0								
0x0FE8	WREG	7:0								
0x0FE9	FSR0	15:8								
0x0FEB	PLUSW0	7:0								
0x0FEC	PREINC0	7:0								
0x0FED	POSTDEC0	7:0								
0x0FEE	POSTINC0	7:0								
0x0FEF	INDR0	7:0								
0x0FF0	Reserved									
0x0FFB	PCL	7:0								
0x0FF9	PCLAT	7:0								
0x0FFA	PCLATH	15:8								
0x0FFC	STKPTR	7:0								
0x0FFD	TOS	15:8								
0x0FFD	TOSU	23:16								

Ejemplo

- Si ejecutamos "goto inicio" en el siguiente programa:

El mismo código de abajo pero sin etiquetas:

```

org 0x0000
goto init_conf
org 0x0020
init_conf: clr TRISD
inicio:    comf PORTB, W
          movwf LATD
          goto inicio
end
  
```

0x0000
0x0020
0x0022
0x0024
0x0026

hace un salto a 0x0020

haz un salto a 0x0022
modifica el PC

Direcciones de la memoria de programa

```

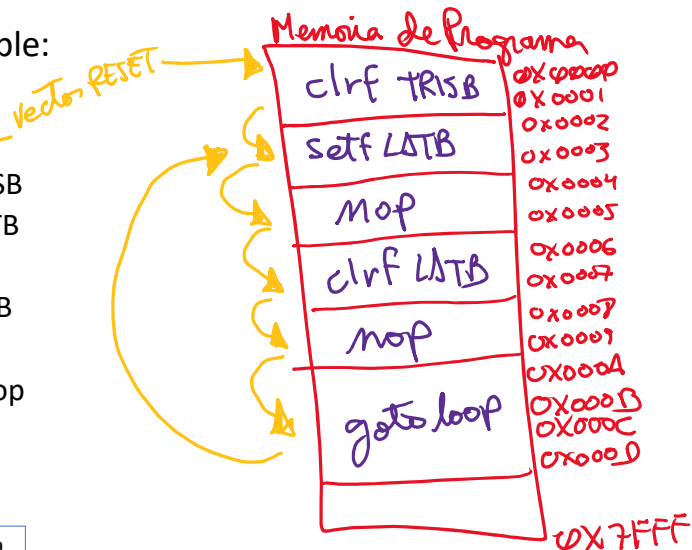
org 0x0000
goto 0x0020
org 0x0020
init_conf: clr TRISD
inicio:    comf PORTB, W
          movwf LATD
          goto 0x0022
end
  
```

¿Cómo funciona el PC?

- Un programa simple:

```

org 0x0000
inicio:  clr TRISB
loop:    setf LATB
         nop
         clr LATB
         nop
         goto loop
end
  
```



Nota: PC va de dos en dos, nunca será impar

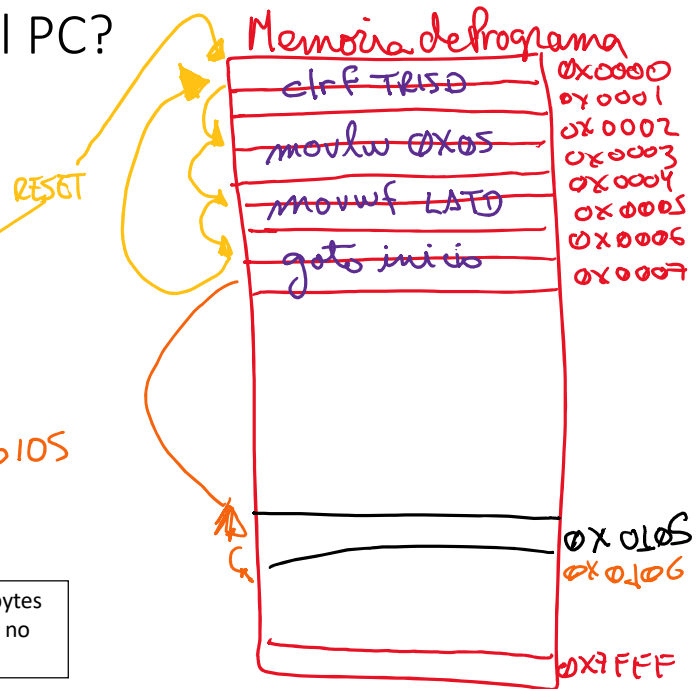
¿Cómo funciona el PC?

- Se tiene el siguiente programa

```

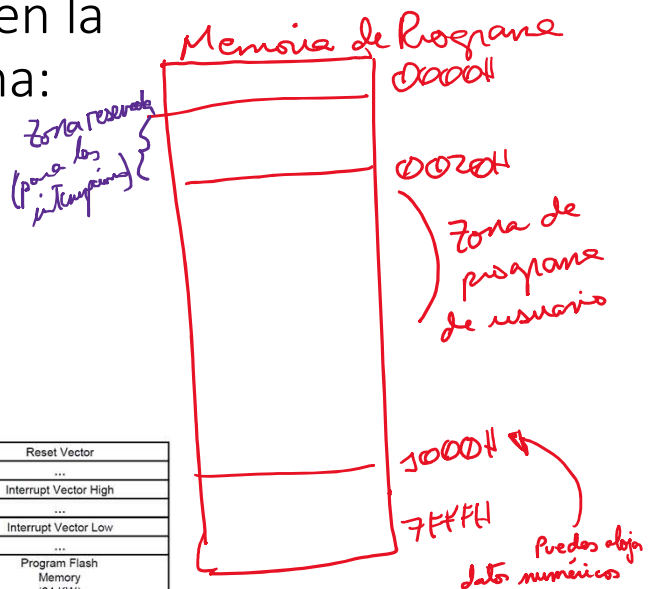
org 000000H
inicio: clrf TRISD
        movlw 05H
        movwf LATD, 1
        bra inicio
        end
  
```

Nota: Las instrucciones en MPASM ocupan 2 bytes
 Nota: Se comprueba que PC va de dos en dos, no puede contener una dirección impar



Información alojada en la memoria de programa:

- En la memoria de programa podemos alojar no solamente instrucciones de un programa, sino también datos que serán constantes (no se podrán modificar cuando el microcontrolador entre en operación)



00 0000h	Reset Vector
...	...
00 0008h	Interrupt Vector High
...	...
00 0018h	Interrupt Vector Low
...	...
00 001Ah to 01 FFFFh	Program Flash Memory (64 KW)
02 0000h to 1F FFFFh	Not Present ⁽²⁾
20 0000h	

Otro ejemplo de PC:

- Las últimas 4 instrucciones emulan un salto a una posición de memoria, como si fuera una instrucción *bra* o *goto*
- No es recomendable usar el PC para hacer saltos si es que no tienes cuidado**

bra 86H

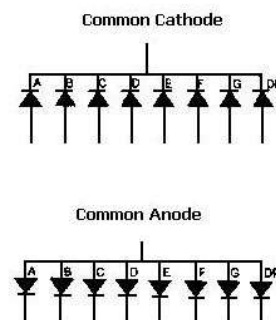
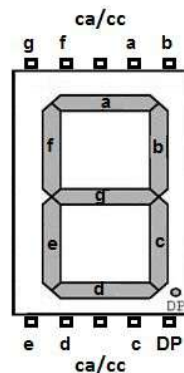
```

ORG 000000H
bra 000080H           ;00H

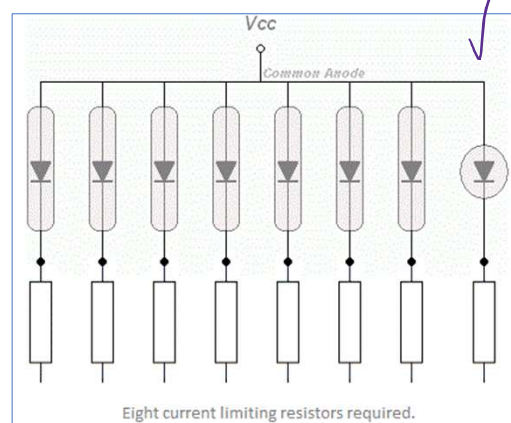
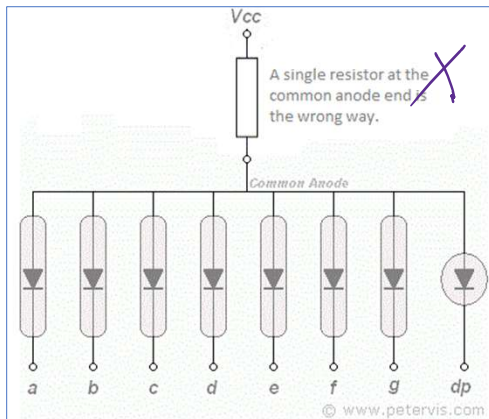
ORG 000080H
movlb 4H              ;80H
bcf TRISA, 0, 1       ;82H
bcf ANSELA, 0, 1      ;84H
btg LATA, 0, 1        ;86H
nop                   ;88H
clrf PCLATU           ;
clrf PCLATH           ;
movlw 86H              ;
movwf PCL              ; hace un salto a 86H
  
```

El display de siete segmentos

- Dos tipos: ánodo común y cátodo común
- Tablas de decodificación diferentes entre ellos.
- Es necesario colocarle resistencias limitadoras de corriente para cada segmento que compone el display (100ohm a 1kohm).**
- Evitar usar solo una resistencia en el pin común del display ya que al cambiar de carácter mostrado se tendrá un efecto de cambio de intensidad luminosa.



Recomendación sobre la conexión del display



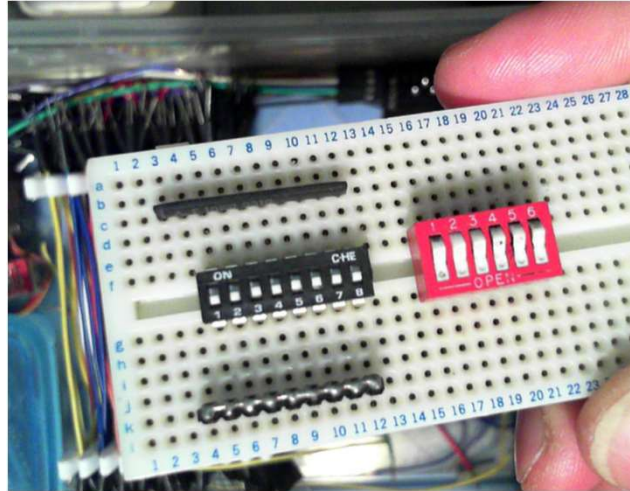
- Aplica también para la configuración de cátodo común

Los precursores a los displays de siete segmentos: Tubos Nixie



Uso de DIP Switches y Chip Resistors

- Nos ayudan a hacer el proceso de prototipaje mas rápido, ordenado y simple



Ejemplo 2024-1 Sem4

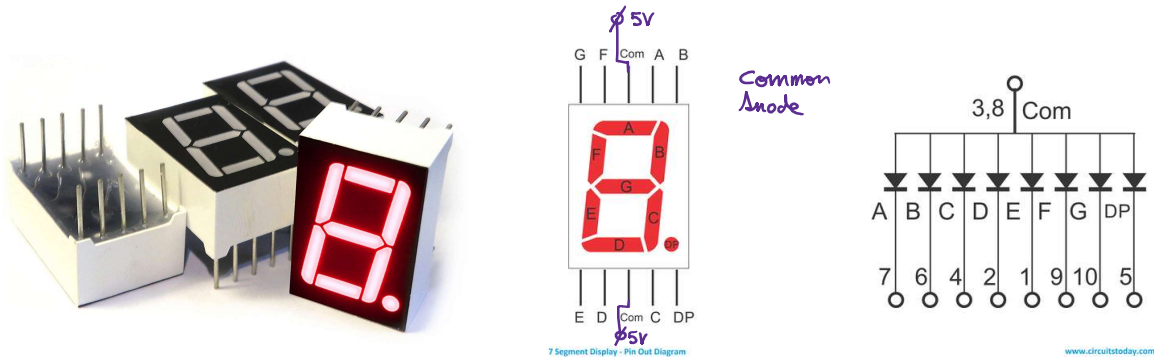
```

1  PROCESSOR 18F57Q43
2  #include "cabecera.inc"
3
4  PSECT upcino, class=CODE, reloc=2, abs
5  upcino:
6      ORG 000000H
7      bra configuro
8
9      ORG 001000H
10  tabla_7s:  DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H
11
12      ORG 000100H
13  configuro:
14      movlb 0H
15      movlw 60H
16      movwf OSCCON1, 1
17      movlw 03H
18      movwf OSCFRQ, 1
19      movlw 40H
20      movwf OSCEN, 1
21      movlb 4H
22      movlw 0FFH
23      movwf TRISB, 1
24      movlw 0F0H
25      movwf ANSELB, 1
26      movlw 0FH
27      movwf WPUB, 1
28      movlw 80H
29
30      movwf TRISD, 1
31      movwf ANSELD, 1
32      movlw 00H
33      movwf TBLPTRU, 1
34      movlw 10H
35      movwf TBLPTRH, 1
36      movlw 00H
37      movwf TBLPTRL, 1
38
39  inicio:
40      comf PORTB, 0, 1      ;complemento PORTB y lo mando a Wreg
41      andlw 0FH            ;enmascaramiento para que pase solo los 4 menos sign
42      movwf TBLPTRL, 1     ;cambio de direccion de apunte de TBLPTR segun entra
43      TBLRD*              ;accion de lectura del TBLPTR
44      movff TABLAT, LATD   ;mando el contenido leido al puerto del display
45      bra inicio
46  end upcino

```

Observación:

- Los pines denominados “comunes” (Com) son el mismo nodo, es la misma conexión!



Desarrollo de la tabla de decodificación BCD para display de 7 segmentos cátodo común:

CC	X	SG	SF	SE	SD	SC	SB	SA	HEX	
	RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0		
0	0	0	1	1	1	1	1	1	0x3F	
1	0	0	0	0	0	1	1	0	0x06	
2	0	1	0	1	1	0	1	1	0x5B	
3	0	1	0	0	1	1	1	1	0x4F	
4	0	1	1	0	0	1	1	0	0x66	
5	0	1	1	0	1	1	0	1	0x6D	
6	0	1	1	1	1	1	0	1	0x7D	
7	0	0	0	0	0	1	1	1	0x07	
8	0	1	1	1	1	1	1	1	0x7F	
9	0	1	1	0	0	1	1	1	0x67	

Circuitos TTL decodificadores de siete segmentos

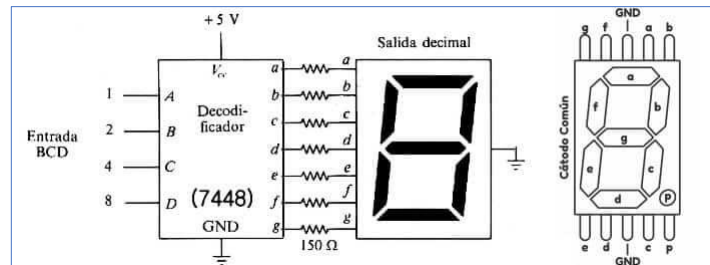
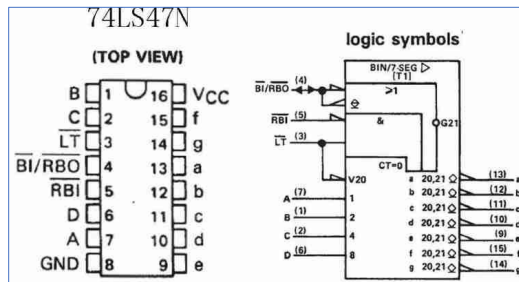
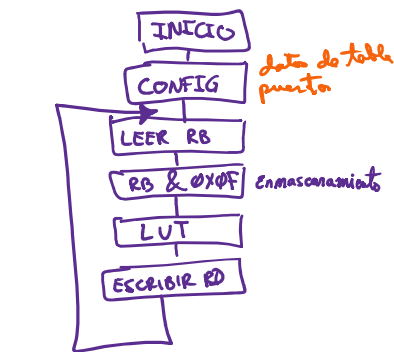
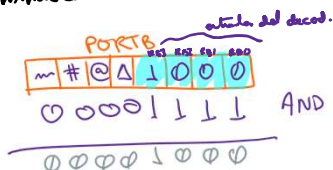


Diagrama de flujo



Enmascaramiento



Código previo (empleando PC como LUT)

```

31 loop:
32     movf PORTB, w
33     andlw 0x0F
34     movwf valor_entrada
35     call tabla_pc
36     movwf LATD
37     goto loop
38
39 tabla_pc:
40     movf valor_entrada, w
41     addwf PCL, f
42     (0)retlw 0x3F
43     (1)retlw 0x06
44     (2)retlw 0x5B
45     (3)retlw 0x4F
46     retlw 0x66
47     retlw 0x6D
48     retlw 0x7D
49     retlw 0x07
50     retlw 0x7F
51     retlw 0x67
52
53     end

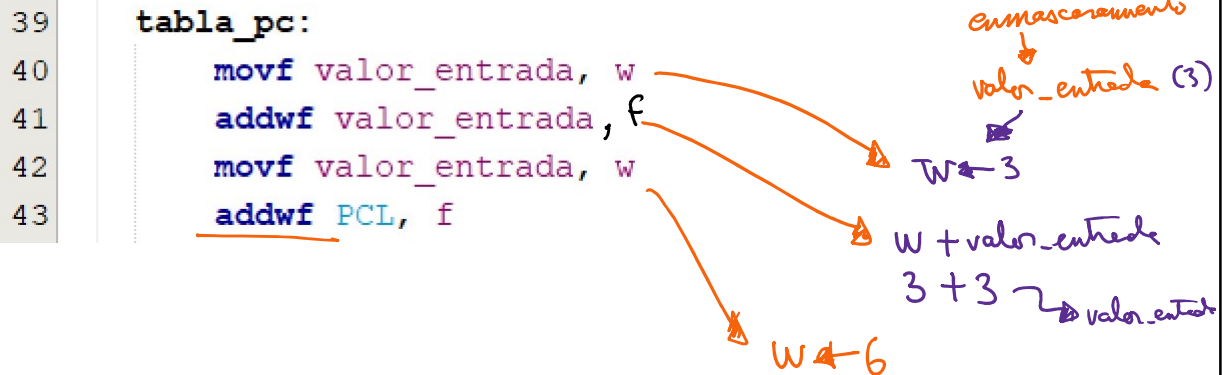
```

Mem. prog

movf val..., w	x + 0
addwf PCL, f	x + 1
retlw 0x3F	x + 2
retlw 0x06	x + 3
retlw 0x5B	x + 4
retlw 0x4F	x + 5
retlw 0x66	x + 6
retlw 0x6D	x + 7
retlw 0x7D	x + 8
retlw 0x07	x + 9
retlw 0x7F	:
retlw 0x67	:

Problema: Debido a que se esta empleando el PC como LUT, al interactuar los datos de entrada (PORTB) con PC se obtendrán saltos a direcciones **impares**!

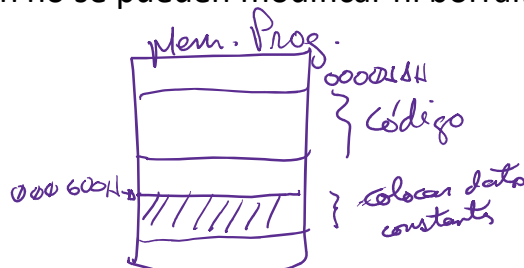
Propuesta de arreglo de problema



Nota: Para que se realicen los saltos correctos empleando el PC (como LUT) se propone que luego de obtener el valor del dato de entrada de PORTB, éste dato se sumará a si mismo de tal modo que sea el doble, se obtenga las direcciones de salto en número par para el PC y funcione correctamente la LUT

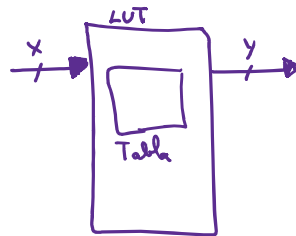
Acerca de la memoria de programa

- En esta memoria no solamente permite el alojamiento de instrucciones.
- También se puede alojar datos constantes, estos datos constantes solo se graban en un evento de programación del microcontrolador, luego en operación no se pueden modificar ni borrar.



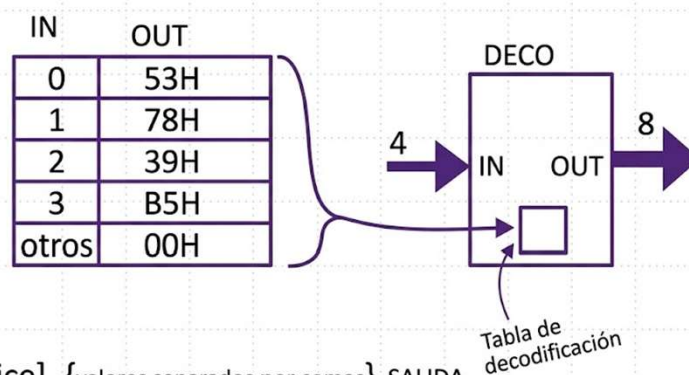
Tablas de búsqueda (lookup tables - LUT)

- Decodificadores implementados en software



- Dos maneras de implementar LUT en XC8 PIC Assembler de PIC18 Q10
 - Utilizado la funcionalidad del PC (**preferible no usar**)
 - Utilizando el TBLPTR

¿Cómo funciona la tabla de búsqueda?

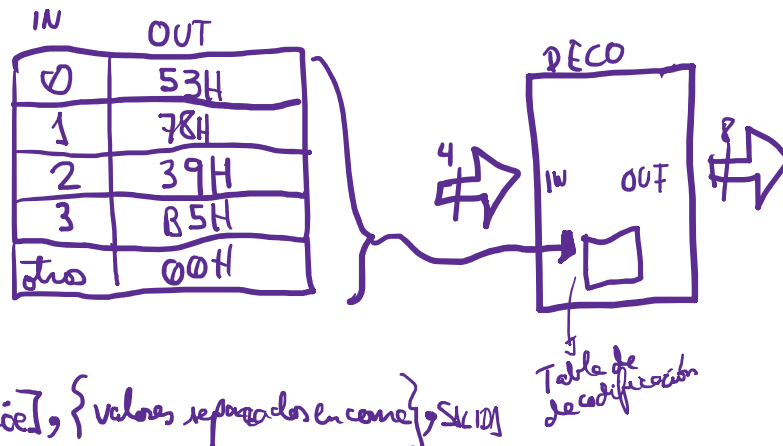


LOOKUP [indice], {valores separados por comas}, SALIDA

LOOKUP 2, {53H, 78H, 39H, B5H}, SALIDA

↳ SALIDA = 39H

¿Cómo funciona la tabla de búsqueda?



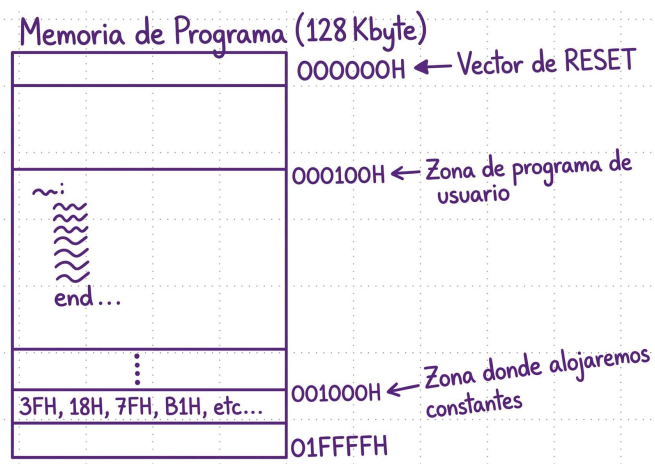
LOOKUP [índice], {valores referenciados en memoria}, SALIDA

LOOKUP 2, {53H, 78H, 39H, B5H}, SALIDA

→ SALIDA = 39H

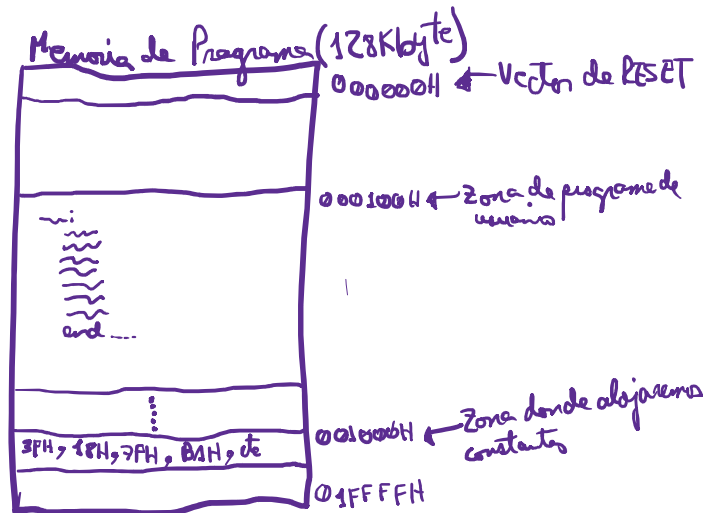
¿En dónde guardo los datos de la tabla de decodificación?

- En la memoria de programa, alejado de la zona donde está tu programa:



¿En dónde guardo los datos de la tabla de decodificación?

- En la memoria de programa, alejado de la zona donde está tu programa:



¿Cómo funciona la tabla de búsqueda?

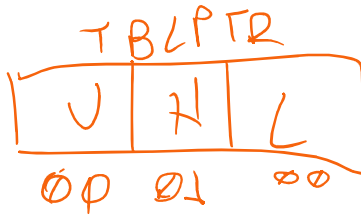
```
//indice: 0    1    2    3    4    5    6
unsigned char tablota[]={0x10, 0x35, 0x44, 0x55, 0xBE, 0xAF, 0x99};
unsigned char salida = 0;

void main(void){
    salida = tablota[6];           //al ejecutarse salida=99H
}
```

```
tablota = [0x10, 0x35, 0x44, 0x55, 0xBE, 0xAF, 0x99]
salida = tablota(3);
```


Operación con el TBLPTR (Table Pointer ó Puntero de Tabla)

- En este programa se busca obtener el contenido de 102H en la memoria y trasladarlo al puerto D



```

1      PROCESSOR 18F4550
2      #include "cabecera.inc"
3
4      PSECT principal, class=CODE, reloc=2, abs
5
6      principal:
7          ORG 0100H
8      numeros:  DB  7FH, 0AAH, 5AH, 4EH, 33H
9
10         ORG 0000H
11         goto configuro
12         ORG 0020H
13
14     configuro:
15         clrf TRISD      ;Puerto D como salida
16         clrf TBLPTRU
17         movlw HIGH numeros
18         movwf TBLPTRH
19         movlw LOW numeros
20         movwf TBLPTRL   ;TBLPTR apuntando a 000100H
21
22     loop:
23         movlw 02H
24         addwf TBLPTRL, 1 ;Dirección de apunte modificado a 000102H
25         TBLRD*          ;Leo contenido apuntado por TBLPTR
26         movf TABLAT, 0   ;Muevo el contenido de TABLAT hacia WReg
27         movwf LATD       ;Muevo contenido de WReg hacia LATD
28         goto loop
29     end principal

```

Modificación del ejemplo del decodificador anterior empleando TBLPTR

```

1      #include "cabecera.inc"
2
3      PSECT rstVect, class=CODE, reloc=2, abs
4
5
6      ORG 0500H
7      cadena:  DB  0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x67, 0x79, 0x79, 0x79, 0x79, 0x79
8
9
10     ORG 0000H
11     rstVect:  goto configuracion
12
13     ORG 0020H
14     configuracion:  movlw 80H
15                     movwf TRISD      ;RD6:RD0 como salidas
16                     clrf TBLPTRU
17                     movlw HIGH cadena
18                     movwf TBLPTRH
19                     movlw LOW cadena
20                     movwf TBLPTRL   ;Asignamos direccion a TBLPTR (500H)
21
22     inicio:      movf PORTE, w
23                     andlw 0FH
24                     movwf TBLPTRL
25                     TBLRD*
26                     movff TABLAT, LATD
27                     goto inicio
28
29     end rstVect

```

- Como segunda opción para implementar una LUT es empleando el puntero de tabla (TBLPTR) donde accederá a determinado dato ubicado en la memoria de programa dependiendo de la dirección asignada.

Ejemplo:

- Desarrollar un programa donde se tenga almacenado los siguientes datos en la **memoria de programa**:
 - 00500H: 04H
 - 00501H: AFH
 - 00502H: BEH
 - 00503H: 89H
- Elaborar un algoritmo que permita leer los datos anteriores y arrojarlas de manera secuencial a través de RD con periodo de NOP

Desarrollo del ejemplo:

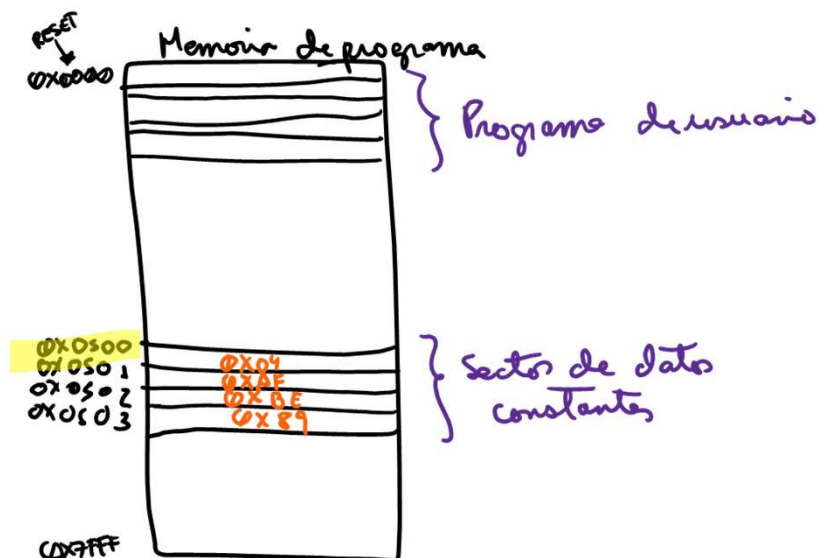
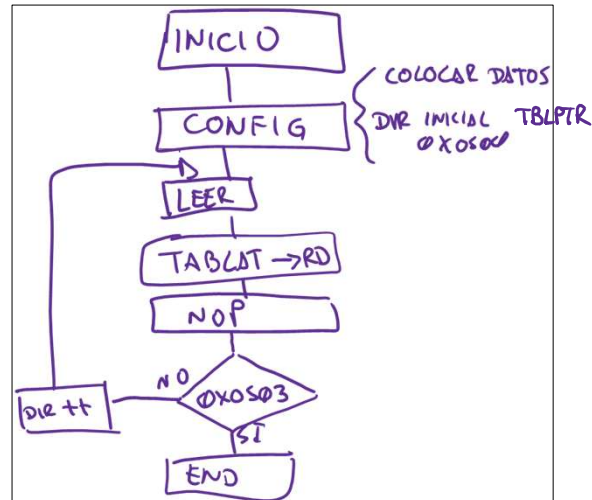


Diagrama de flujo:



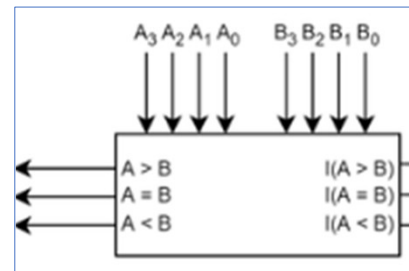
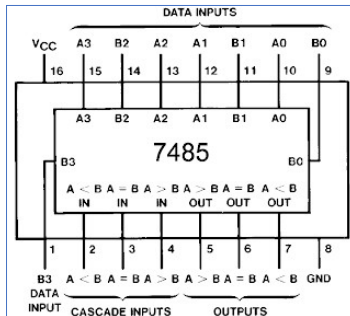
Resumen de instrucciones con toma de decisión:

Formato:

- | | |
|---------------------------------|---|
| • btfss, btfsc | -Prueba el #bit de [registro] si es 0 ó 1 |
| • decfsz, incfsz | -Decrementa o incrementa [registro] y pregunta si es cero. |
| • dcfsnz, icfsnz | -Decrementa o incrementa [registro] y pregunta si no es cero |
| • cpfsgt, cpfseq, cpfslt | - Comparaciones numéricas entre [registro] y W |
| • tstfsz | -Prueba [registro] y salta si es cero |
| • Saltos Branch | -Saltos condicionales |

Recordando al 7485 (comparador de magnitud de 4 bits)

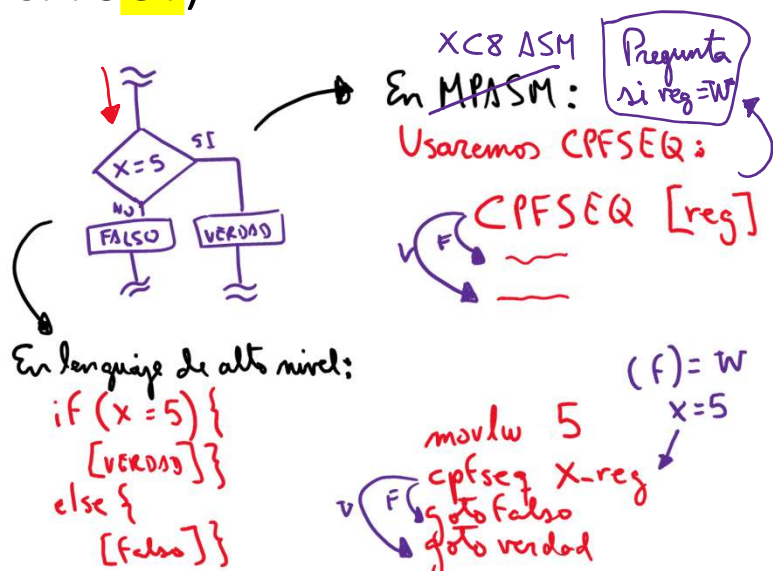
- Compara dos números binarios de 4 bits



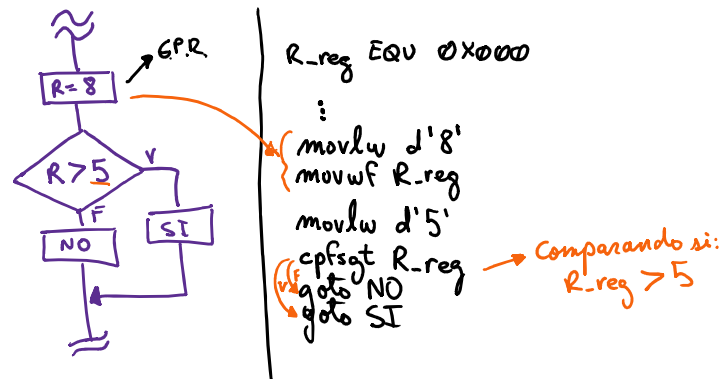
Instrucciones de comparación numérica (CPFSEQ, CPFSLT, CPFSGT)

$\text{CPFSEQ} \rightarrow f = W_{\text{reg}}$
 $\text{CPFSLT} \rightarrow f < W_{\text{reg}}$
 $\text{CPFSGT} \rightarrow f > W_{\text{reg}}$

Nota: El valor a comparar debe de estar previamente en Wreg



Ejercicio: Pasar a XC8 ASM el siguiente diagrama de flujo:



Nota: En las instrucciones $cplsgt$ el segundo parámetro de la comparación debe de estar en Wreg

Ejercicio:

- Implementar un comparador de magnitud de dos números de 8 bits:

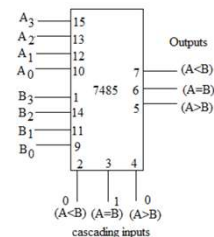
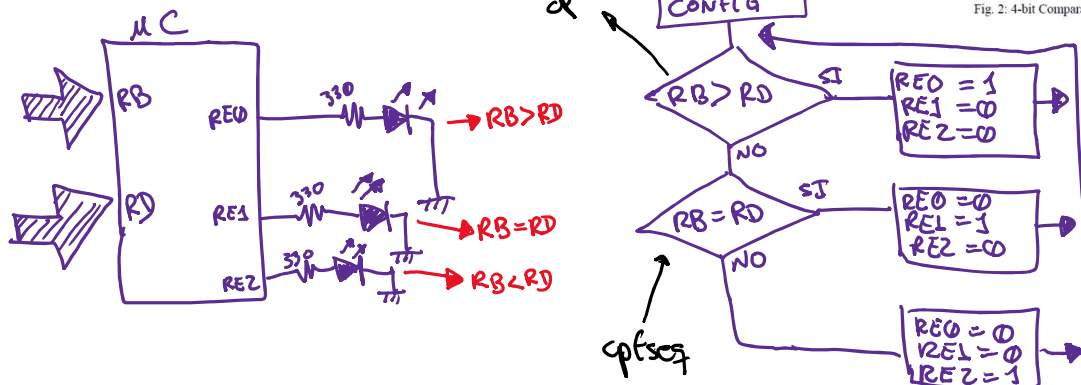
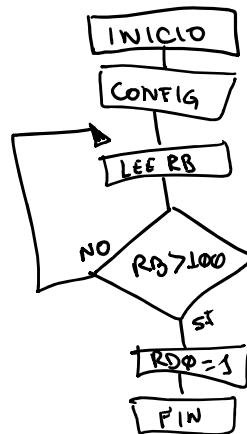
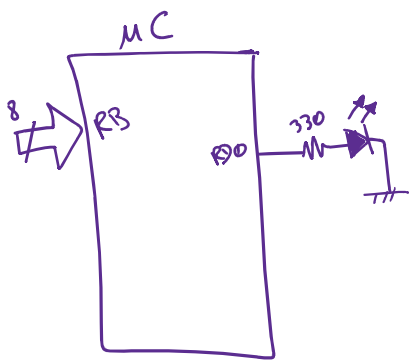


Fig. 2: 4-bit Comparator

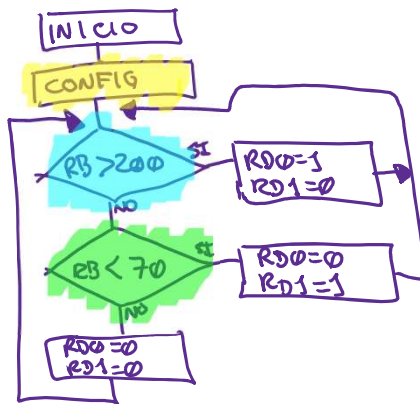
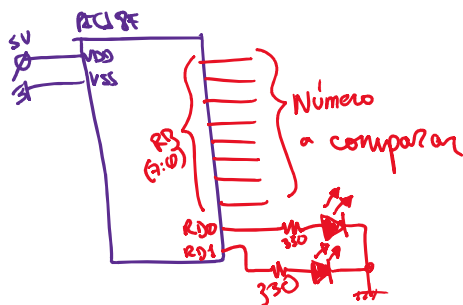
Ejercicio: Leer el valor de RB y colocar a uno el puerto RD0 únicamente cuando dicho valor sea mayor de 100.



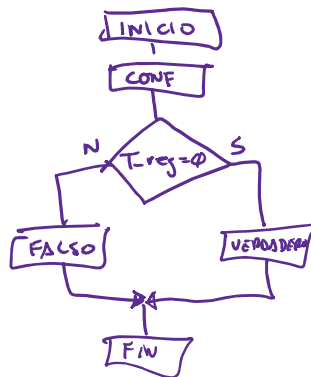
$f > w_{reg}$
 bucle:
 movlw .100
 cpfsgt PORTB
 goto bucle
 bsf LATD, 0
 ...

Ejercicio:

- Desarrollar un programa para que compare lo que se esta ingresando en RB y arroje lo siguiente: RD0=1 cuando RB>200 y RD1=1 cuando RB<70, cuando no se cumplan las dos condiciones las dos salidas permanecerán en cero.



Ejemplo: Pregunta si T_reg (GPR) es igual a cero, si es cierto va a etiqueta verdadero, si no es cierto va a etiqueta falso



Opción 1:

```

inicio: movlw .0
        cplseg T-reg
        goto FALSO
        goto VERDADERO
  
```

Opción 2:

```

inicio: tstfsz T-reg
        goto FALSO
        goto VERDADERO
  
```

Opción 3:

```

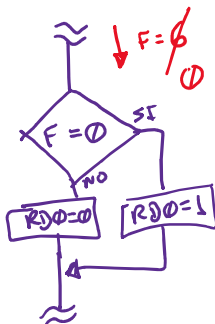
inicio: movf T-reg, W
        sublw .0
        btfss STATUS, Z
        goto FALSO
        goto VERDADERO
  
```

Opción 4:

```

inicio: movf T-reg, W
        sublw .0
        bz VERDADERO
FALSO:  ≡
VERDADERO: ≡
  
```

Aplicación de sublw para tomas de decisión:

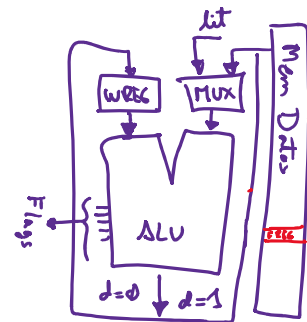


```

movf F-reg, 0
sublw 00H
btfss STATUS, Z
  
```

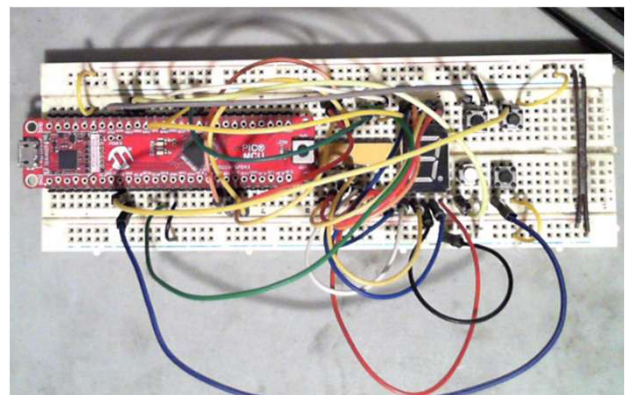
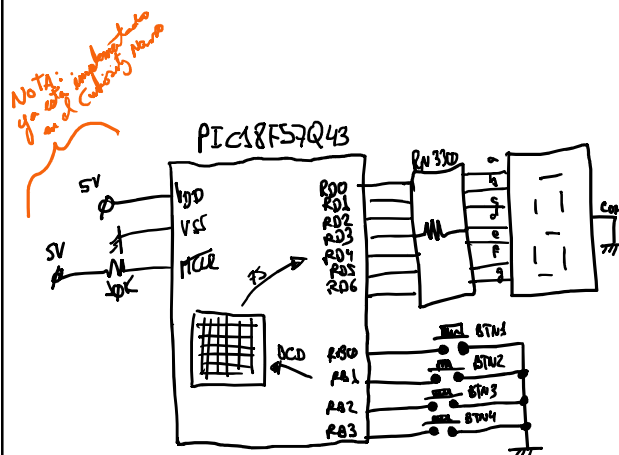
bit 2
(Z)

$$\begin{array}{r} 0 - \\ F-reg \\ \hline 0 - \\ 6 \\ \hline -6 \end{array}$$
 ¿Se levantará algún flag?
 $Z=0 \quad N=1$



- | | | |
|------|---|------------------------|
| BC | n | Branch if Carry |
| BN | n | Branch if Negative |
| BNC | n | Branch if Not Carry |
| BNN | n | Branch if Not Negative |
| BNOV | n | Branch if Not Overflow |
| BNZ | n | Branch if Not Zero |
| BOV | n | Branch if Overflow |
| BRA | n | Branch Unconditionally |
| BZ | n | Branch if Zero |

Ejemplo 2024-1 Sem4

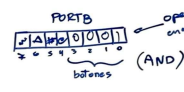
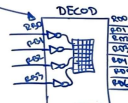


Ejemplo 2024-1 Sem4

Ejemplo de aplicación: Decodificador BCD-7segCC

Observaciones del hardware:

- Entradas del decodificador: RB0 ~ RB3
son activos en bajo
- Salidas del decodificador: RD0 ~ RD6
salidas digitales con pull-up activo



⇒ Tenemos que hacer dos operaciones antes de hacer el proceso de la tabla de decodificación

- complemento
- enmascaramiento

Realizando "LOOK UP TABLES" o tablas de búsqueda.

```
uint_8 tabla[] = {0x3F, 0x06, 0x5B, 0x4F, ...};
uint_8 salida;
```

```
salida = tabla[(~PORTB) & 0x0F];
```

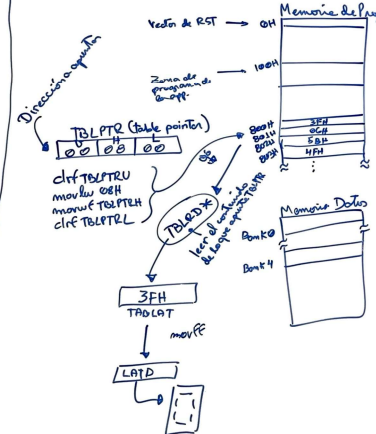
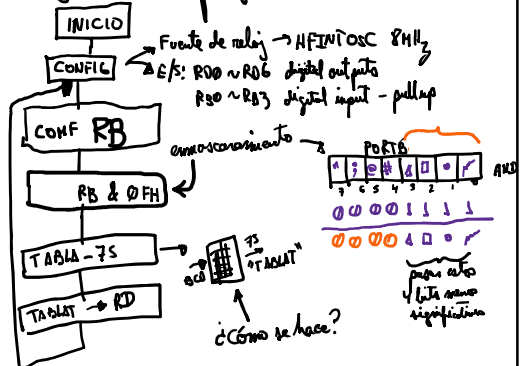


Diagrama de flujo:



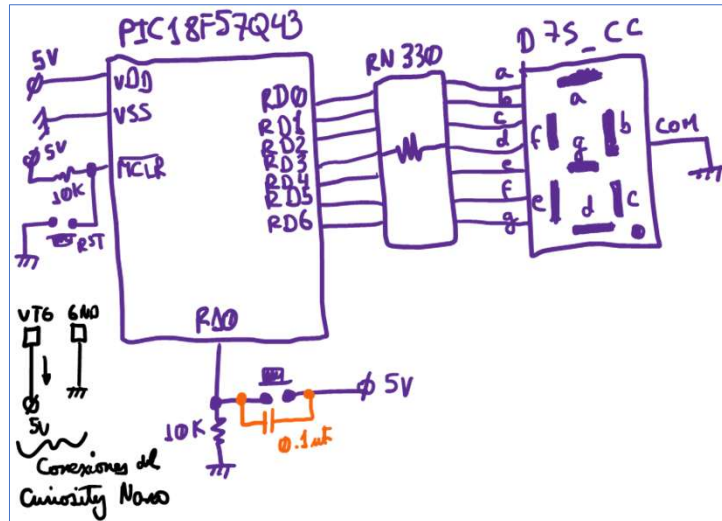
Ejemplo 2024-1 Sem4

```
1  PROCESSOR 18F57Q43
2  #include "cabecera.inc"
3
4  PSECT upcino, class=CODE, reloc=2, abs
5  upcino:
6  ORG 000000H
7  bra configur0
8
9  ORG 001000H
10 tabla_7s: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H
11
12 ORG 000100H
13 configur0:
14 movlb 0H
15 movlb 60H
16 movwf OSCCON1, 1
17 movlw 03H
18 movwf OSCFRQ, 1
19 movlw 40H
20 movwf OSCEN, 1
21 movlb 4H
22 movlw 0FFH
23 movwf TRISB, 1
24 movlw 0F0H
25 movwf ANSELB, 1
26 movlw 0FH
27 movwf WPUB, 1
28 movlw 80H
```

```
29 movwf TRISD, 1
30 movwf ANSEL, 1
31 movlw 00H
32 movwf TBLPTRU, 1
33 movlw 10H
34 movwf TBLPTRH, 1
35 movlw 00H
36 movwf TBLPTRL, 1
37
38 inicio:
39 comf PORTB, 0, 1 ;complemento PORTB y lo mando a Wreg
40 andlw 0FH ;enmascaramiento para que pase solo los 4 menos sign
41 movwf TBLPTRL, 1 ;cambio de direccion de apunte de TBLPTR segun entra
42 TBLRD* ;accion de lectura del TBLPTR
43 movff TABLAT, LATD ;mando el contenido leído al puerto del display
44 bra inicio
45
46 end upcino
```

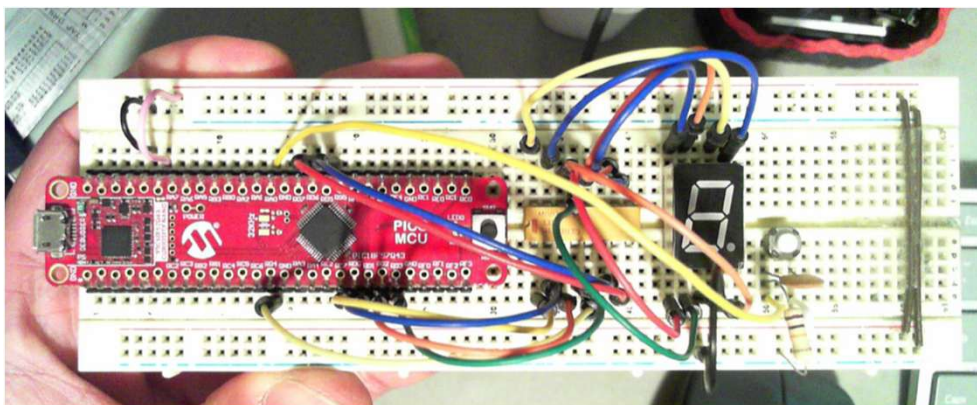
Ejemplo 2024-2

- Desarrollar un contador 0-9 con una entrada de cuenta activa en alto y visualización en display de siete segmentos del tipo cátodo común.



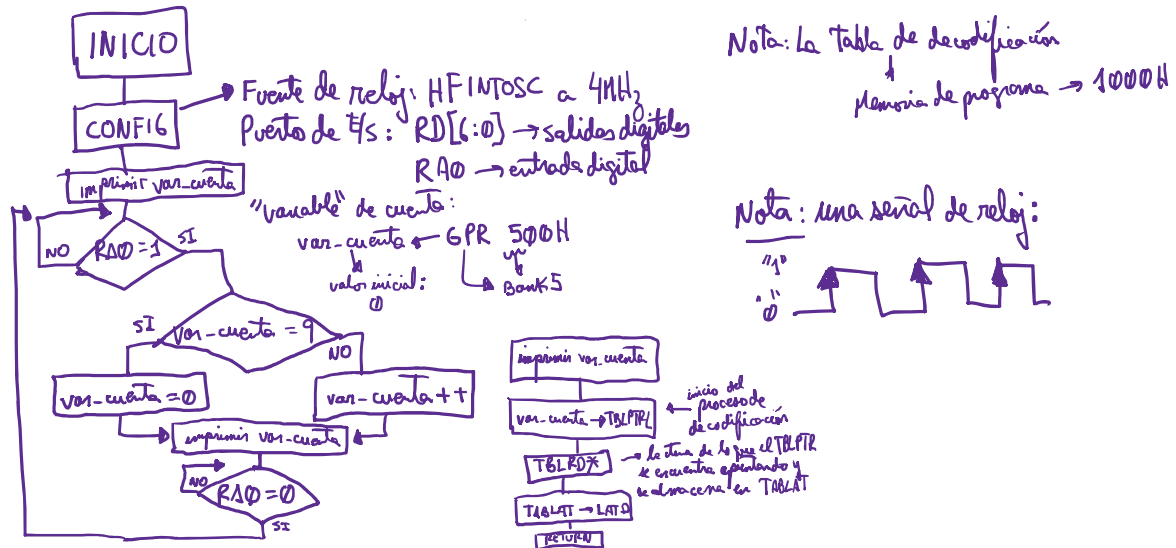
Ejemplo 2024-2

- Desarrollar un contador 0-9 con una entrada de cuenta activa en alto y visualización en display de siete segmentos del tipo cátodo común.



Ejemplo 2024-2

- Desarrollar un contador 0-9 con una entrada de cuenta activa en alto y visualización en display de siete segmentos del tipo cátodo común.



Ejemplo 2024-2

- Desarrollar un contador 0-9 con una entrada de cuenta activa en alto y visualización en display de siete segmentos del tipo cátodo común.

```

1  PROCESSOR 18F57Q43
2  #include "cabecera.inc"
3
4  var_cuenta EQU 500H ;un label al GPR 500H
5
6  PSECT upcino, class=CODE, reloc=2, abs
7  upcino:
8  ORG 000000H
9  bra configur0
10
11  ORG 000800H
12  decod_bod: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H
13  ; 0 1 2 3 4 5 6 7 8 9
14
15  ORG 000100H
16  configur0:
17  ;configuración de la fuente de reloj
18  movlb 0H
19  movlw 60H
20  movwf OSCCON1, 1
21  movlw 02H
22  movwf OSCFRCQ, 1
23  movlw 40H
24  movwf OSCEN, 1
25  ;configuración de las E/S
26  movlb 4H
27  movlw 80H
28  movwf TRISD, 1 ;RD[6:0] como salidas
29  movwf ANSEL, 1 ;RD[6:0] como digital
30  bsf TRISA, 0, 1 ;RA0 como entrada
31  bcf ANSELA, 0, 1 ;RA0 como digital

```

```

32  ;configuraciones adicionales
33  clrf TBLPTRU, 1
34  movlw 08H
35  movwf TBLPTRH, 1
36  clrf TBLPTRL, 1 ;TBLPTR direccionado a 0800H en la mem de p
37  movlb 5H
38  clrf var_cuenta, 1 ;var_cuenta con valor inicial cero
39  movlb 4H
40  call imprimir_var_cuenta
41
42  inicio_contador:
43  btfsc PORTA, 0, 1 ;pregunto si presione boton
44  bra inicio_contador ;falso, no presione boton
45  movlb 5H ;verdad
46  movlw 9
47  cpfseq var_cuenta, 1 ;pregunto si var_cuenta=9
48  bra no_es_nuevo ;falso
49  bra si_es_nuevo ;verdad
50
51  no_es_nuevo:
52  incf var_cuenta, 1, 1 ;incremento var_cuenta
53  bra siguiente
54
55  si_es_nuevo:
56  clrf var_cuenta, 1 ;mando a cero var_cuenta
57
58  siguiente:
59  call imprimir_var_cuenta
60
61  aun_no:
62  btfsc PORTA, 0, 1 ;pregunto si dejaste de presionar boton
63  bra aun_no
64  bra inicio_contador
65
66  imprimir_var_cuenta:
67  movlb 5H
68  movf var_cuenta, 0, 1 ;nuevo var_cuenta a Wreg
69  movlb 4H
70  movwf TBLPTRL, 1 ;direccion del TBLPTR
71  TBLRD+ ;accion de lectura
72  movff TABLAT, LATD
73  return
74
75  end upcino

```

<pre> 1 PROCESSOR 18F57Q43 2 #include "cabecera.inc" 3 4 PSECT upcino, class=CODE, reloc=2, abs 5 upcino: 6 ORG 000000H 7 bra configur0 8 9 ORG 001000H 10 deco7s: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H 11 ; 0 1 2 3 4 5 6 7 8 9 12 13 ORG 000100H 14 configur0: 15 ;conf fuente de reloj 16 movlb 0H ;voy al Bank0 17 movlw 60H 18 movwf OSCCON1, 1 19 movlw 02H 20 movwf OSCFRO, 1 21 movlw 40H 22 movwf OSCEN, 1 23 ;conf puertos E/S 24 movlb 4H ;voy al Bank4 25 setf TRISB, 1 ;RB[3:0] como entradas 26 movlw 0F0H 27 movwf ANSELB, 1 ;RB[3:0] como digitales 28 bsf TRISE, 0, 1 ;RE0 como entrada 29 bcf ANSELB, 0, 1 ;RE0 como digital 30 movlw 80H 31 movwf TRISD, 1 ;RD[6:0] como salida 32 movwf ANSELD, 1 ;RD[6:0] como digital </pre>	Códigos de ejemplos 2024-2 Decodificador BCD-7Segmentos <pre> 33 ;conf adicional 34 clrf TBLPTRU, 1 35 movlw 10H 36 movwf TBLPTRH, 1 ; U H L 37 clrf TBLPTRL, 1 ;TBLPTR apuntando a 001000H 38 39 inicio_deco: 40 movf PORTB, 0, 1 ;Leo el puerto RB[3:0] y lo almaceno en Wreg 41 andlw 0FH ;Enmascaramiento de los cuatro LSB de Wreg 42 movwf TBLPTRL, 1 ;asignamos el valor de Wreg hacia TBLPTRL 43 TBLRD* ;accion de lectura de lo apuntado por TBLPTR 44 movff TABLAT, LATD ;mueve contenido leído y alojado en TABLAT hacia RD 45 bra inicio_deco 46 47 end upcino </pre>
---	--

<pre> 1 PROCESSOR 18F57Q43 2 #include "cabecera.inc" 3 4 PSECT upcino, class=CODE, reloc=2, abs 5 upcino: 6 ORG 000000H 7 bra configur0 8 9 ORG 001000H 10 decod: DB 3FH, 06H, 5BH, 4FH, 66h, 6DH, 7DH, 07H, 7FH, 67H 11 ; 0 1 2 3 4 5 6 7 8 9 12 13 ORG 000100H 14 configur0: 15 ;configuracion de la fuente de reloj 16 movlb 0H 17 movlw 60H 18 movwf OSCCON1, 1 19 movlw 02H 20 movwf OSCFRO, 1 21 movlw 40H 22 movwf OSCEN, 1 23 ;configurac0n de las E/S 24 movlb 4H 25 movlw 80H 26 movwf TRISB, 1 ;RB[6:0] como salida 27 movwf ANSELB, 1 ;RB[6:0] como digitales 28 setf TRISD, 1 ;RD como entradas 29 movlw 0F0H 30 movwf ANSELD, 1 ;RD[3:0]] como digital </pre>	Códigos de ejemplos 2024-2 Decodificador BCD-7Segmentos <pre> 31 ;configuraciones adicionales 32 clrf TBLPTRU, 1 33 movlw 10H 34 movwf TBLPTRH, 1 35 clrf TBLPTRL, 1 ;direccion asignada a TBLPTR=001000H 36 37 inicio_decod: 38 ;tengo que leer el puerto de entrada RD[3:0] 39 movf PORTD, 0, 1 ;leo RD y lo mando a Wreg 40 andlw 0FH ;enmascaramiento a Wreg con el valor 0FH 41 ;tengo que decodificar usando el TBLPTR 42 movwf TBLPTRL, 1 ;muevo contenido de Wreg a TBLPTRL 43 TBLRD* ;lectura de lo que apunta TBLPTR y se graba en TABLAT 44 ;tengo que escribir TABLAT a RB[6:0] 45 movff TABLAT, LATB ;muevo contenido de TABLAT a RB 46 bra inicio_decod 47 48 end upcino </pre>
---	--

Códigos de ejemplos 2024-2

- Decodificador BCD-7Segmentos

Códigos de ejemplos 2024-2

- Decodificador BCD-7Segmentos

1	PROCESSOR 18F57Q43		34	bcf ANSEL, 0, 1 ;RE0 como digital	
2	#include "cabecera.inc"		35	;configuraciones adicionales	
3			36	clrf TBLPTRU, 1	
4	cuenta EQU 500H ;asignacion de label a GPR 500H		37	movlw 10H	
5			38	movwf TBLPTRH, 1	
6	PSECT upcino, class=CODE, reloc=2, abs		39	clrf TBLPTRL, 1 ;direccion asignada a TBLPTR=001000H	
7	upcino:		40	movlb 5H ;al bank 5	
8	ORG 000000H		41	clrf cuenta, 1 ;valor inicial de cuenta=0	
9	bra configuro		42	movlb 4H ;al bank 4	
10			43	call imp_7s	
11	ORG 001000H		44		
12	decod: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H		45	inicio_contador:	
13	; 0 1 2 3 4 5 6 7 8 9		46	btfss PORTE, 0, 1 ;pregunto si presione el boton	
14			47	bra inicio_contador ;falso, no presione el boton	
15	ORG 000100H		48	movlb 5H	
16	configuro:		49	movlw 9	
17	;configuracion de la fuente de reloj		50	cpfseq cuenta, 1 ;pregunto si cuenta=9	
18	movlb 0H		51	bra no_es_nueve	
19	movlw 60H		52	bra si_es_nueve	
20	movwf OSCCON1, 1		53	no_es_nueve:	
21	movlw 02H		54	incf cuenta, 1, 1 ;incremento cuenta	
22	movwf OSCFRCQ, 1		55	bra juntada	
23	movlw 40H		56	si_es_nueve:	
24	movwf OSCEN, 1		57	clrf cuenta, 1 ;cuenta igual a cero	
25	;configuracion de las E/S		58	juntada:	
26	movlb 4H		59	call imp_7s	
27	movlw 80H		60	movlb 4H	
28	movwf TRISE, 1 ;RB[6:0] como salidas		61	btfsc PORTE, 0, 1 ;pregunto si solte el boton	
29	movwf ANSELB, 1 ;RB[6:0] como digitales		62	bra juntada ;falso, sigue pulsado	
30	setf TRISD, 1 ;RD como entradas		63	bra inicio_contador	
31	movlw 0F0H				
32	movwf ANSELD, 1 ;RD[3:0] como digitales				
33	bsf TRISE, 0, 1 ;RE0 como entrada				
			65	imp_7s:	
			66	movlb 5H	
			67	movf cuenta, 0, 1 ;mando cuenta a Wreg	
			68	movlb 4H	
			69	movwf TBLPTRL, 1 ;nuevo Wreg a TBLPTRL	
			70	TBLRD*	
			71	movff TABLAT, LATB	
			72	return	
			73		
			74	end upcino	

Códigos de ejemplos 2024-2

- Contador BCD-7Segmentos

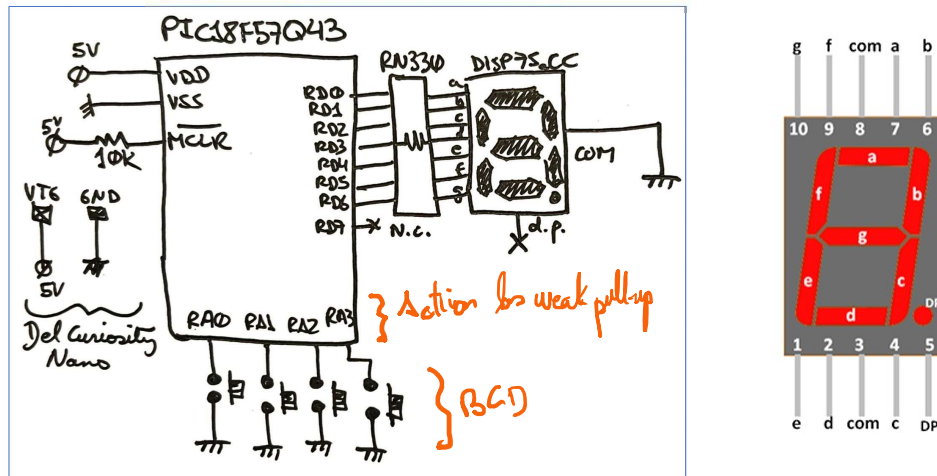
1	PROCESSOR 18F57Q43		34	;conf adicional	
2	#include "cabecera.inc"		35	clrf TBLPTRU, 1	
3			36	movlw 10H	
4	var_cuenta EQU 500H ;asignando etiqueta a GPR 500H		37	movwf TBLPTRH, 1	
5			38	clrf TBLPTRL, 1 ;TBLPTR esta apuntando a 001000H	
6	PSECT upcino, class=CODE, reloc=2, abs		39	movlb 5H ;nos vamos al Bank5	
7	upcino:		40	clrf var_cuenta, 1 ;valor inicial de var_cuenta=0	
8	ORG 000000H		41	movlb 4H ;nos vamos a Bank4	
9	bra configuro		42	call imp_7s ;llamo a subrutina imp_7s	
10			43	inicio conta:	
11	ORG 001000H		44	btfss PORTE, 0, 1 ;pregunto si presione boton	
12	deco_7s: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H		45	bra \$-2 ;falso, retorno a linea anterior	
13			46	movlb 5H ;nos vamos al Bank5	
14	ORG 000100H		47	movlw 9	
15	configuro:		48	cpfseq var_cuenta, 1 ;pregunto si var_cuenta=9	
16	;conf fuente de reloj:		49	bra no_es_nueve ;falso, no es nueve	
17	movlb 0H		50	clrf var_cuenta, 1 ;verdad, es nueve y lo mandamos a cero	
18	movlw 60H		51	bra siguiente ;salto a label siguiente	
19	movwf OSCCON1, 1		52	no_es_nueve:	
20	movlw 02H		53	incf var_cuenta, 1, 1 ;incremento var_cuenta	
21	movwf OSCFRCQ, 1		54	siguiente:	
22	movlw 40H		55	call imp_7s	
23	movwf OSCEN, 1		56	btfsc PORTE, 0, 1 ;pregunto si deje de pulsar boton	
24	;conf puertos E/S:		57	bra \$-2 ;falso, boton sigue presionado	
25	movlb 4H		58	bra inicio_conta ;retorno al inicio	
26	movlw 80H		59		
27	movwf TRISD, 1 ;RD[6:0] como salidas		60	imp_7s:	
28	movwf ANSELD, 1 ;RD[6:0] como digitales		61	movlb 5H ;al Bank5	
29	setf TRISB, 1 ;RB[3:0] como entradas		62	movf var_cuenta, 0, 1 ;leo var_cuenta y lo mando a Wreg	
30	movlw 0F0H		63	movlb 4H ;al Bank4	
31	movwf ANSELB, 1 ;RB[3:0] como digitales		64	movwf TBLPTRL, 1 ;movemos Wreg a TBLPTRL para iniciar decod	
32	bsf TRISE, 0, 1 ;RE0 como entrada		65	TBLRD*	
33	bof ANSELE, 0, 1 ;RE0 como digital		66	movff TABLAT, LATD ;accion de lectura de lo apuntado por TBLPTR	
			67	return	
			68		
			69	end upcino	

Códigos de ejemplos 2024-2

- Contador BCD-7Segmentos

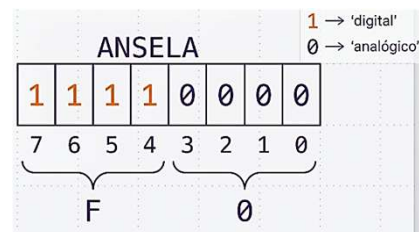
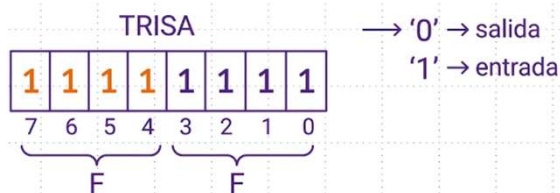
Ejercicio 2025-1

- Desarrollar un decodificador de BCD a siete segmentos para display de siete segmentos de cátodo común donde las entradas son activos en bajo (deberán de activar las weak pullup de los pines)



Configuración de los pines RA

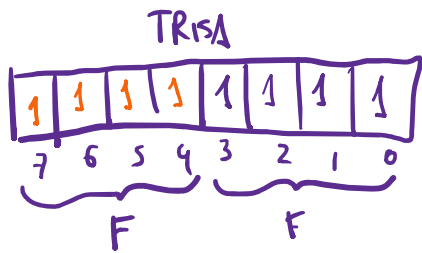
```
movlw 0FFH
movwf TRISA, 1      ;RA3 al RA0 como entradas
movlw 0F0H
movwf ANSELA, 1     ;RA3 al RA0 como digitales
```



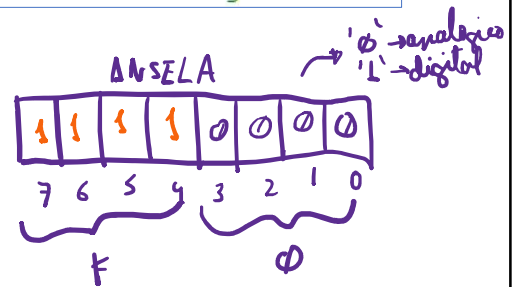
Configuración de los pines RA

```

movlw OFFH
movwf TRISA, 1      ;RA3 al RA0 como entradas
movlw OF0H
movwf ANSELA, 1     ;RA3 al RA0 como digitales
  
```

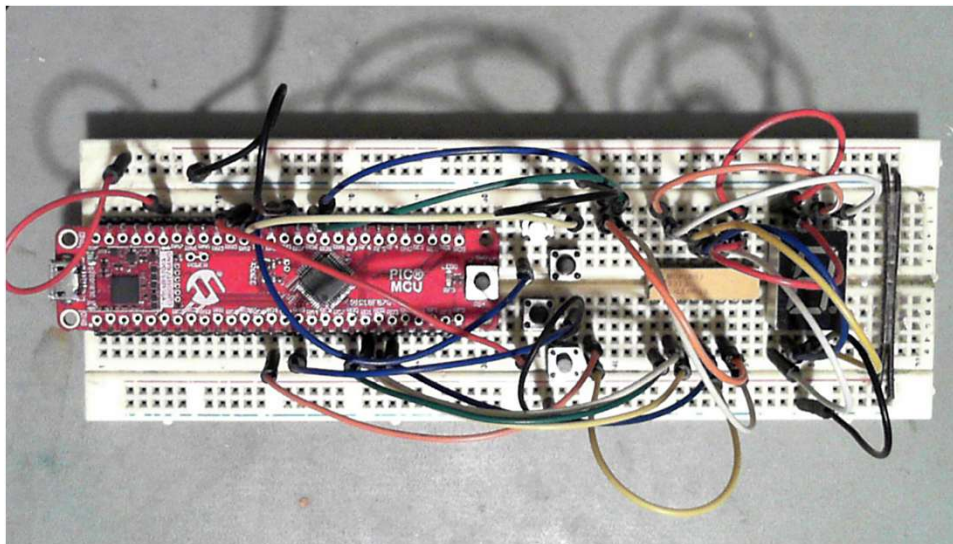


→ 0 → salida
→ 1 → entrada



Ejercicio 2025-1

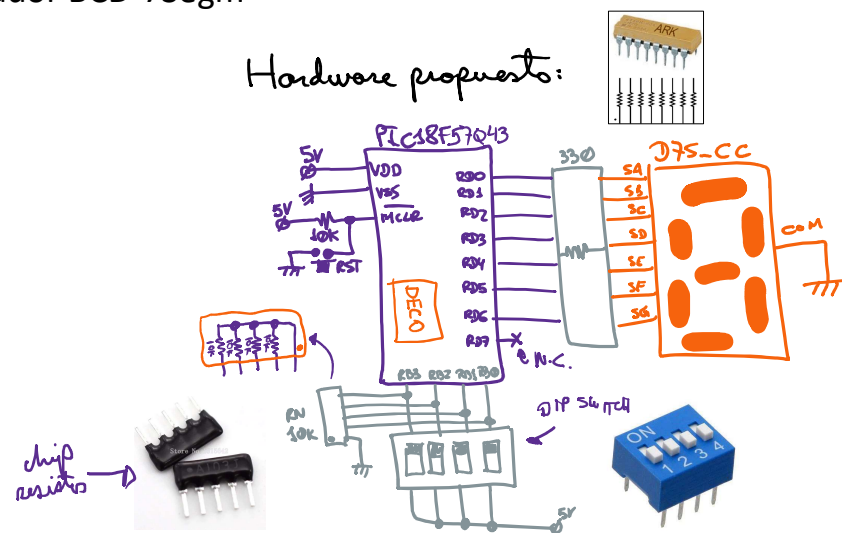
- Implementar el siguiente circuito



-

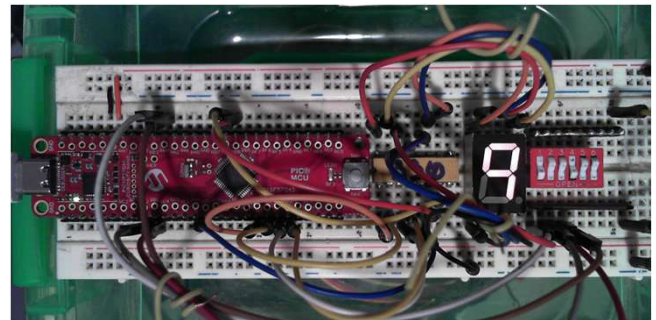
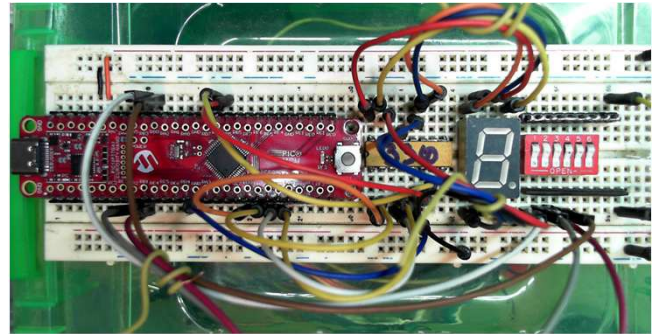
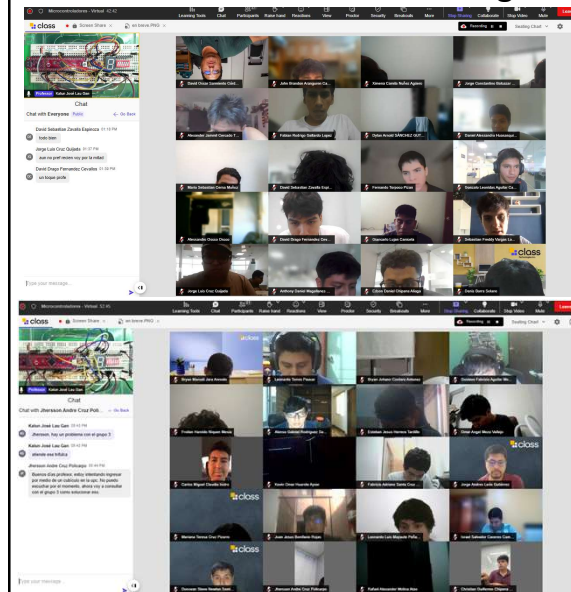
Ejercicio 2026-1

- Hardware properties:



Ejercicio 2026-1

• Decodificador BCD-7Segm



```

1  PROCESSOR 18F57Q43
2  #include "cabecera.inc"
3
4  PSECT upcino, class=CODE, reloc=2, abs
5  upcino:
6      ORG 000000H          ;vector de reset
7      bra configuro
8
9      ORG 000200H
10     ;0 1 2 3 4 5 6 7 8 9
11     tabs_7s: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H
12
13     ORG 000100H          ;zona de user program
14     configuro:
15         ;conf el modulo de oscilador
16         movlb 0H          ;Al Bank0
17         movlw 60H
18         movwf OSCCON1, 1
19         movlw 02H
20         movwf OSCFRQ, 1
21         movlw 40H
22         movwf OSCEN, 1
23         ;conf las E/S
24         movlb 4H          ;Al Bank4
25         movlw 80H
26         movwf TRISD, 1    ;RD6 al RD0 como salidas
27         movwf ANSELD, 1   ;RD6 al RD0 como digitales
28         movlw 0FOH
29         ;Por defecto RB es entrada en un Power on Reset
30         movwf ANSELB, 1   ;RB3 al RB0 como digitales

```

Ejercicio 2026-1

```

31     ;conf el puntero de tabla para que apunte a 200H
32     clrf TBLPTRU, 1
33     movlw 02H
34     movwf TBLPTRH, 1
35     clrf TBLPTRL, 1      ;TBLPTR esta apuntando a 000200H
36
37     inicio:
38         movf PORTE, 0, 1  ;Leo RB y lo almaceno en Wreg
39         andlw 0FH         ;operacion de enmascaramiento AND con Wreg
40         movwf TBLPTRL, 1  ;muevo Wreg a TBLPTRL
41         TBLRD*            ;accion de lectura de lo que apunta TBLPTR
42         movff TABLAT, LATD ;muevo contenido de TABLAT hacia RD
43         bra inicio
44
45     end upcino

```

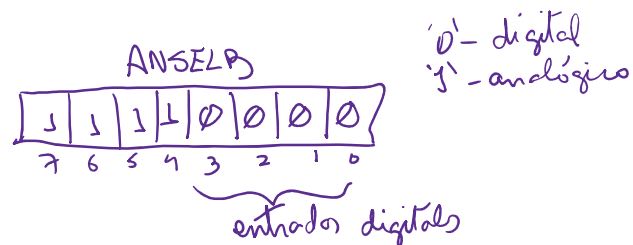
¿Por qué 0F0H en ANSELB?

- El formato hexadecimal indica que si el número inicia con una letra, se le antepone un 0
- Si necesitamos RB3 al RB0 como digitales:



¿Por qué 0F0H en ANSELB?

- El formato hexadecimal indica que si el número inicia con una letra, se le antepone un 0
- Si necesitamos RB3 al RB0 como digitales:



¿Por qué no se declaran las entradas?

- Por defecto en un PoR (Power on Reset), los puertos por defecto se comportan como entradas analógicas

19.14.3 TRISx

Name: TRISx

Tri-State Control Register

Bit	7	6	5	4	3	2	1	0
	TRISx7	TRISx6	TRISx5	TRISx4	TRISx3	TRISx2	TRISx1	TRISx0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	1	1	1	1	1	1	1	1

Bits 0, 1, 2, 3, 4, 5, 6, 7 – TRISxn Port I/O Tri-state Control

Value	Description
1	PORTx output driver is disabled. PORTx pin configured as an input (tri-stated).
0	PORTx output driver is enabled. PORTx pin configured as an output.

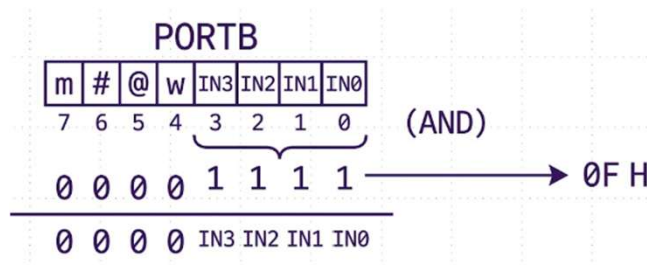


Important:

- The TRIS bit associated with the $\overline{\text{MCLR}}$ pin is read-only and the value is '1'
- Refer to the "Pin Allocation Table" for details about $\overline{\text{MCLR}}$ pin and pin availability per port
- Unimplemented bits will read back as '0'

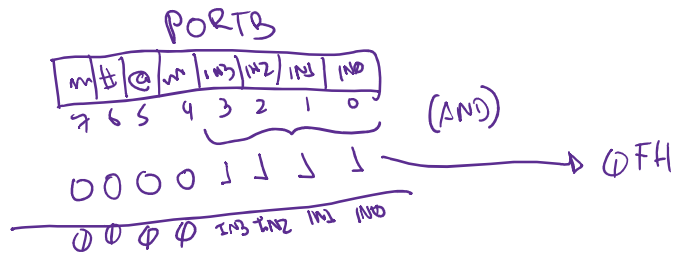
¿Por qué hay que enmascarar los datos del RB?

- Para que solo se contemplen la información entre RB3 y RB0 solamente



¿Por qué hay que enmascarar los datos del RB?

- Para que solo se contemplen la información entre RB3 y RB0 solamente



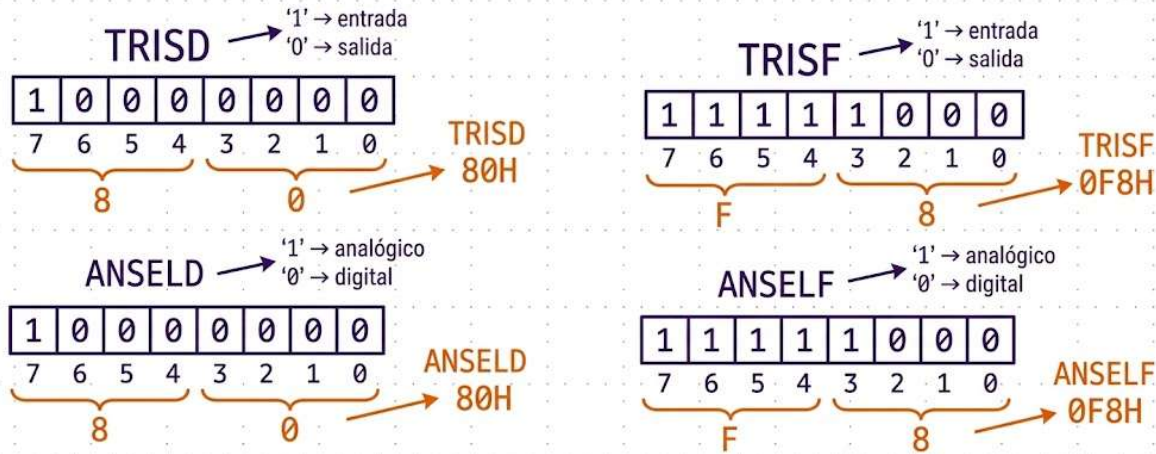
Ejercicio 2026-1

```

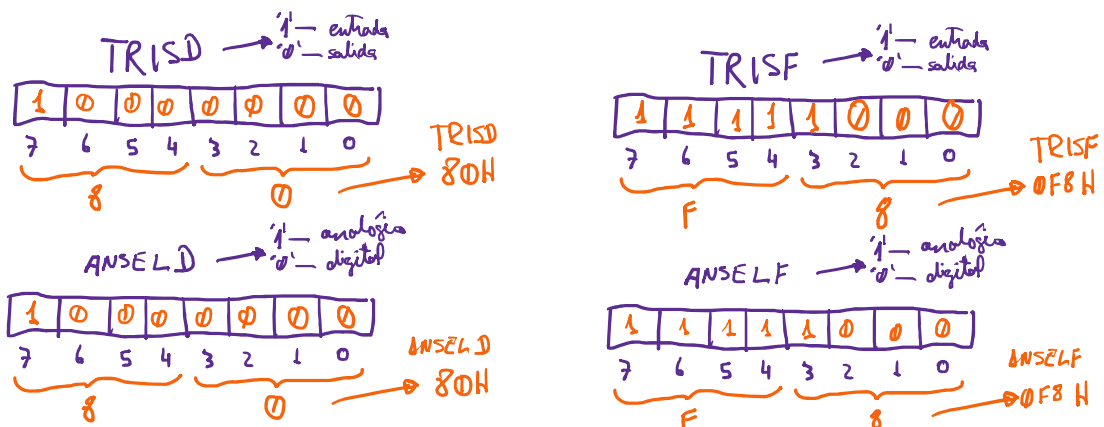
1  PROCESSOR 18F57Q43
2  #include "cabecera.inc"
3
4  PSECT upcino, class=CODE, reloc=2, abs
5
6  upcino:
7  ORG 000000H          ;vector de RESET
8  bra configur0        ;salto a label configur0
9
10 ORG 000300H
11 tablita_7s: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 67H
12
13 ORG 000100H
14 configur0:
15 ;conf el modulo de oscilador
16 movlb 0H              ;Bank0
17 movwf OSCCON1, 1      ;HFINTOSC 1:1
18 movlw 02H
19 movwf OSCFRQ, 1       ;HFINTOSC 4MHz
20 movlw 40H
21 movwf OSCEN, 1        ;HFINTOSC enabled
22 ;conf las E/S
23 movlb 4H              ;Bank4
24 movlw 80H
25 movwf TRISD, 1        ;RD6 al RD0 salidas
26 movwf ANSELD, 1       ;RD6 al RD0 digitales
27 movlw 0F0H
28 movwf ANSELB, 1       ;RB3 al RB0 como digitales
29
30 ;conf el TBLPTR para que apunte a 300H de la mem de prog
31 clrf TBLPTRU, 1
32 movlw 03H
33 movwf TBLPTRH, 1
34 clrf TBLPTRL          ;TBLPTR esta apuntando a 000300H
35
36 inicio:
37 movf PORTE, 0, 1      ;Leo contenido de RB y lo mando a Wreg
38 andlw 0FH             ;aplico enmascaramiento AND a wreg con valor 0FH
39 movwf TBLPTRL, 1      ;muevo Wreg hacia TBLPTRL para que funcione la lookup
40 TBLRD*                 ;accion de lectura del TBLPTR
41 movff TABLAT, LATD     ;muevo contenido de TABLAT hacia RD
42 bra inicio
43 end upcino

```

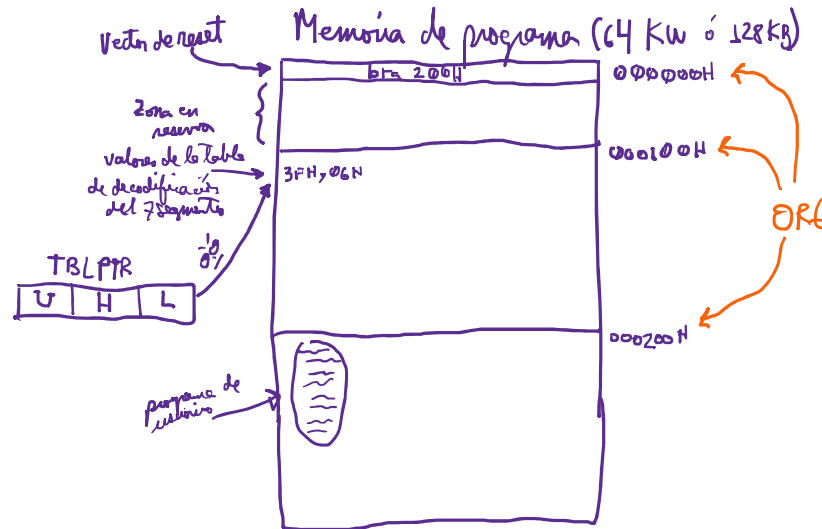
Configuración de E/S mediante valor numérico



Configuración de E/S mediante valor numérico

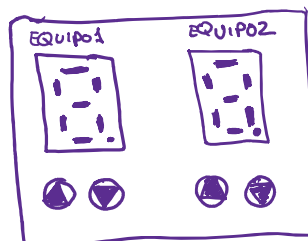


Ubicación de los datos constantes de la tabla de decodificación en la memoria de programa



Ejercicios adicionales:

- Desarrollar un visualizador de mensaje "SOY FELIZ" a través de un display de siete segmentos del tipo cátodo común a razón de una letra a la vez y con periodo de cambio de 500ms.
- Desarrollar un tablero de score para una cancha deportiva, el cual se tenga dos displays y cuatro pulsadores clasificados en un display y dos pulsadores para cada equipo, un pulsador será para incrementar la cuenta y el otro para decrementar la cuenta.



Rutina para obtener los digitos de millar, centena, decena y unidad

- Fuente: MikroE
<https://www.mikroe.com/ebooks/pic-microcontrollers-programming-in-assembly/appendix-b-examples>

Macro "digbyte":

```
digbyte MACRO arg0
    LOCAL Exit0
    LOCAL Exit1
    LOCAL Exit2

    clrf    Dig0
    clrf    Dig1
    clrf    Dig2
    clrf    Dig3

    movf    arg0, W
    movwf   Digtemp
    movlw   .100

Exit2:
    incf    Dig2, f
    subwf   Digtemp, f
    btfsc   STATUS, C
    goto    Exit2
    decf    Dig2, f
    addwf   Digtemp, f

Exit1:
    movlw   .10
    incf    Dig1, f
    subwf   Digtemp, f
    btfsc   STATUS, C
    goto    Exit1
    decf    Dig1, f
    addwf   Digtemp, f

Exit0:
    movf    Digtemp, W
    movwf   Dig0
ENDM
```

Fin de la sesión

- Evaluación LB1 inicia en estos momentos